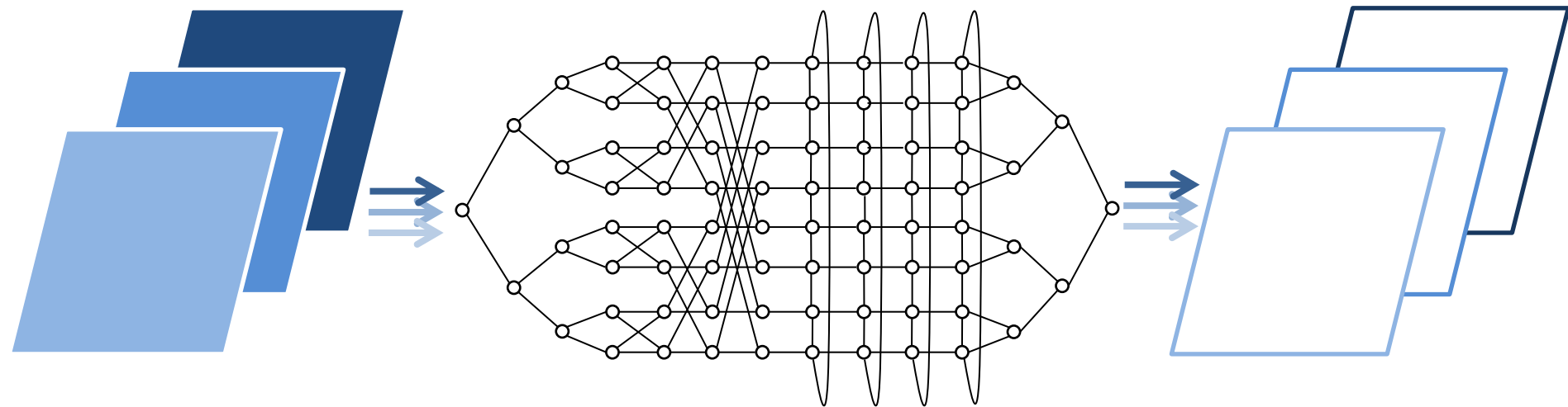


Programmation Parallèle

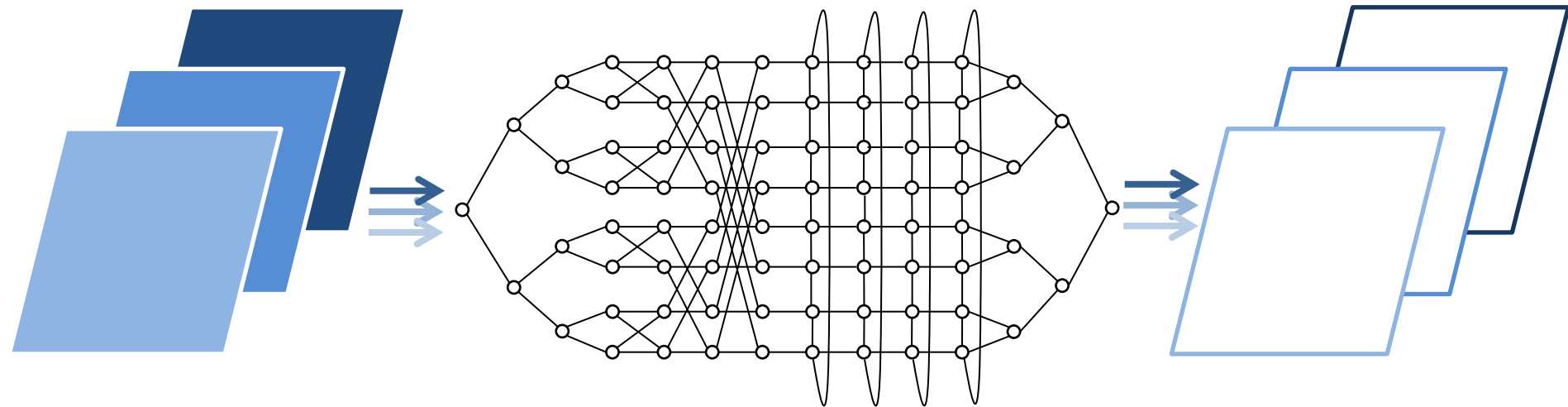
MPI: Message Passing Interface



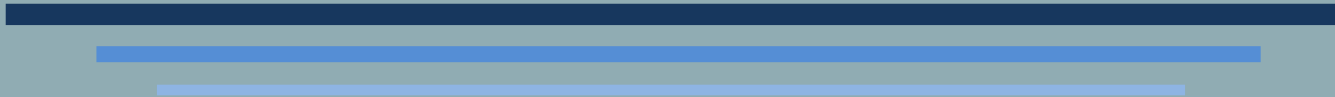
Advanced Collectives

Communications

ENSIIE-HPC/BigData-PP-IPAR-Lecture 4



MPI INTER-COMMUNICATORS



Reminder from course PP-C4



- What we discussed so far are intra-communicators (one group, one context)
- However, when an application is built by composing several parallel modules, it is convenient to allow one module to communicate with another using local ranks for addressing within the second module.
- Inter-communicators are defined to exchange messages between multiple groups

Reminder from course PP-C4



- **Prototype:**

```
int MPI_Intercomm_create(MPI_Comm local_comm, int  
local_leader, MPI_Comm peer_comm, int  
remote_leader, int tag, MPI_Comm *newintercomm)
```

- Processes should provide identical local_comm and local_leader arguments within each group.
- Collective call over the union of both groups

Split

MPI_COMM_WORLD

r_1

r_0

r_3

r_2

r_4

```
MPI_Comm_split(  
MPI_COMM_WORLD,  
r%2, r, &c)
```

Split

MPI_COMM_WORLD

r_1

r_3

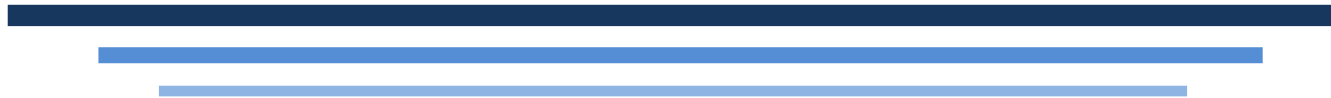
r_0

r_2

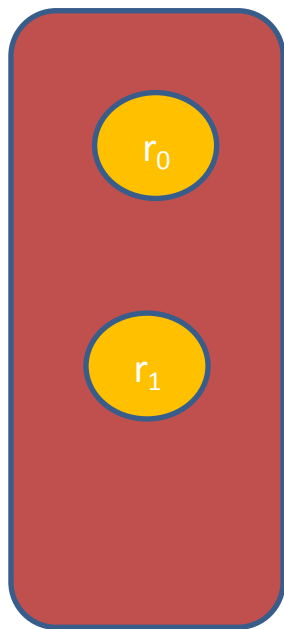
r_4

```
MPI_Comm_split(  
MPI_COMM_WORLD,  
r%2, r, &c)
```

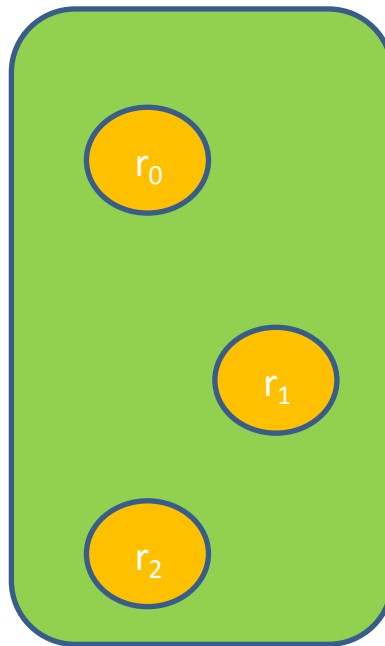
Split



COMM_A



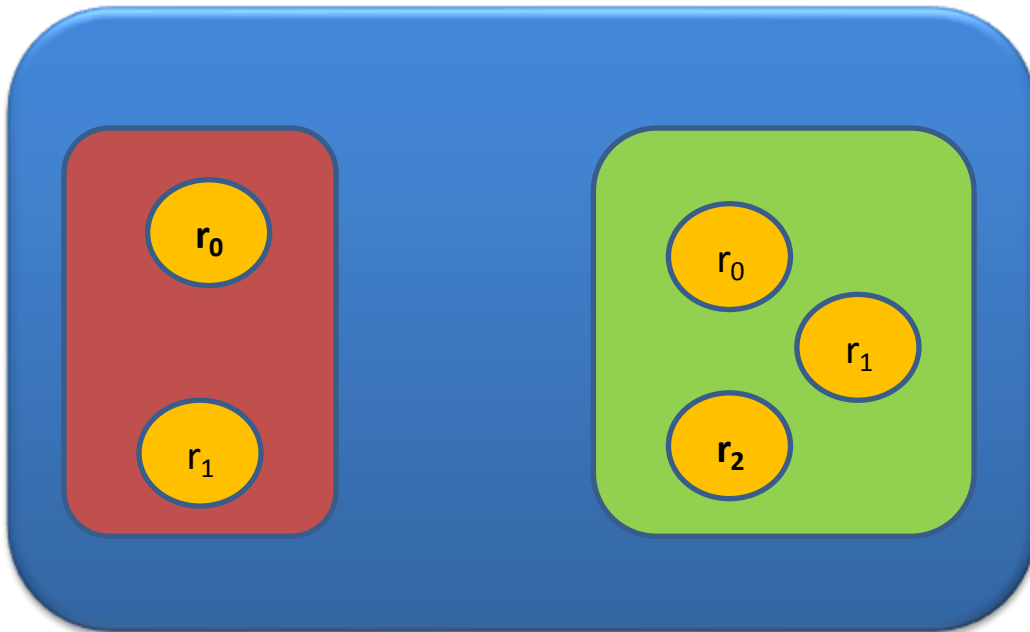
COMM_B



Split

```
MPI_Intercomm_create( COMM_A, 0, COMM_B, 0, my_tag,  
&new_intercomm)
```

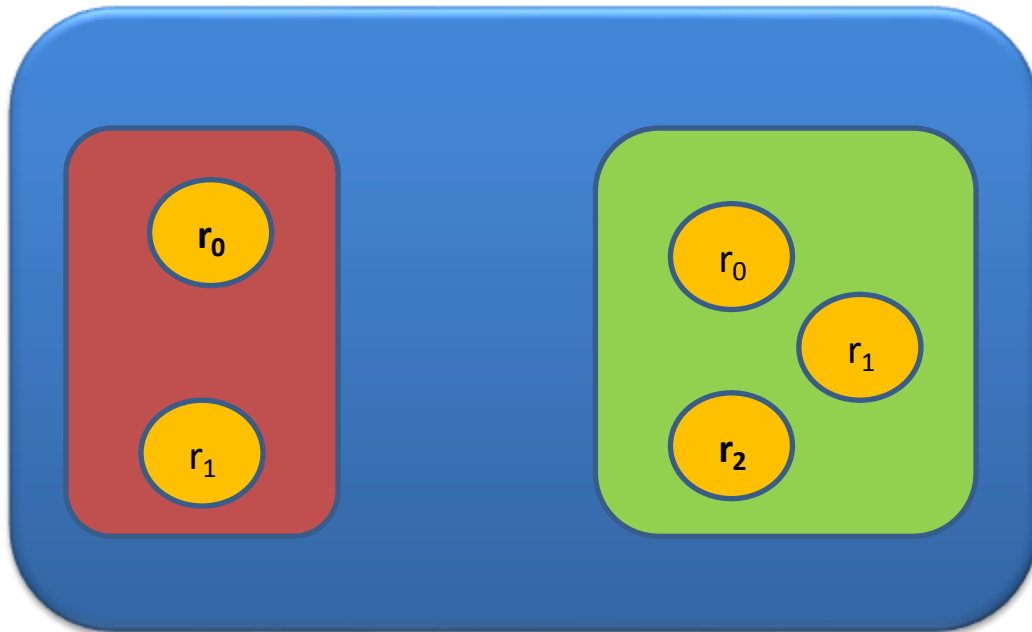
new_intercomm



Split

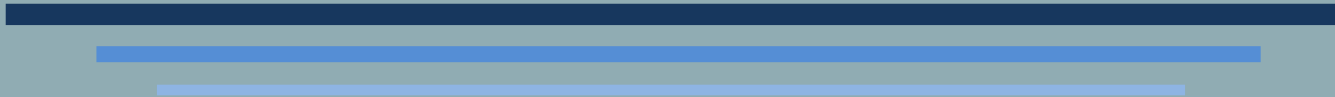
```
MPI_Intercomm_create( COMM_A, 0, COMM_B, 0, my_tag,  
&new_intercomm)
```

new_intercomm



- Rank values do not change
 - Ranks are actually associated with the group of MPI processes, and not the communicator

COLLECTIVES WITH INTERCOMMUNICATORS



Collectives with intercomms.

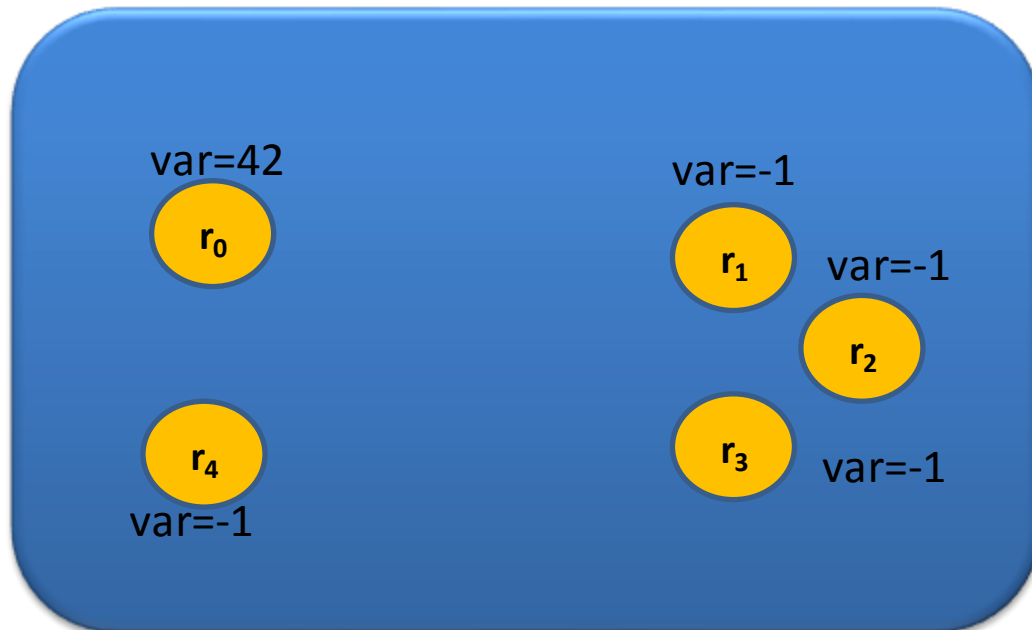
- Communications on intercommunicators use the same functions calls than on intracommunicators
 - `MPI_Bcast(&pi, 1, MPI_FLOAT, root, intracomm);`
 - `MPI_Bcast(&pi, 1, MPI_FLOAT, root, intercomm);`
- However, the behavior is different
- In an intercommunicator, the sender MPI processes and receiver MPI processes are ALWAYS in separate groups

Collectives with intercomms.

- Communications on intercommunicators use the same functions calls than on intracommunicators
 - `MPI_Bcast(&pi, 1, MPI_FLOAT, root, intracomm);`
 - `MPI_Bcast(&pi, 1, MPI_FLOAT, root, intercomm);`
- However, the behavior is different
- In an intercommunicator, the sender MPI processes and receiver MPI processes are ALWAYS in separate groups
- What does it mean ?
 - Example on Bcast

Bcast intracomm

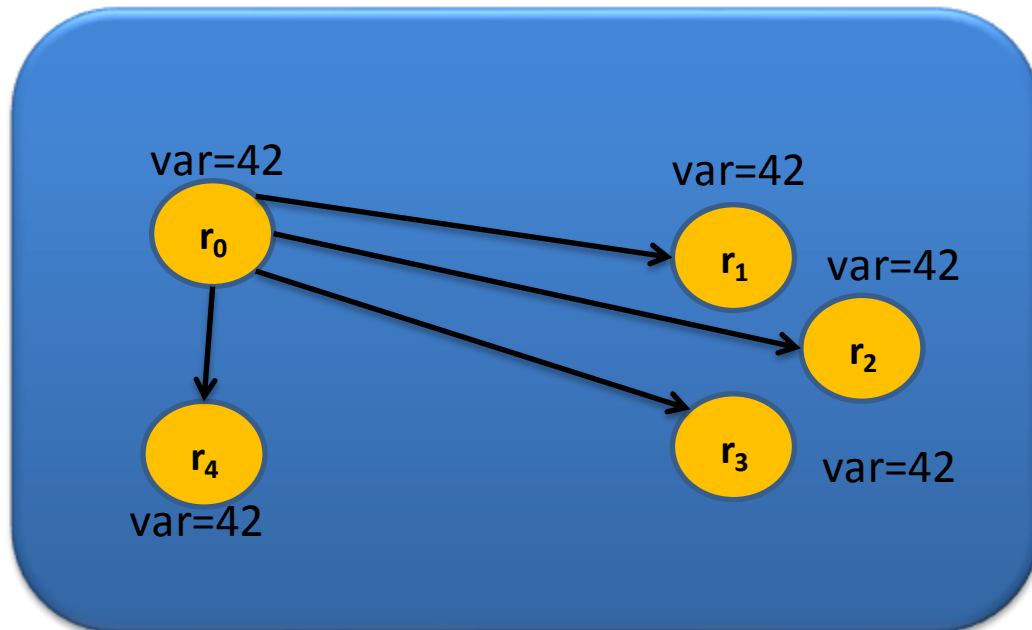
intracomm



Bcast intracommunicator

- A Bcast on an intracommunicator, the root “sends” its data to all other MPI processes in the group of the communicator

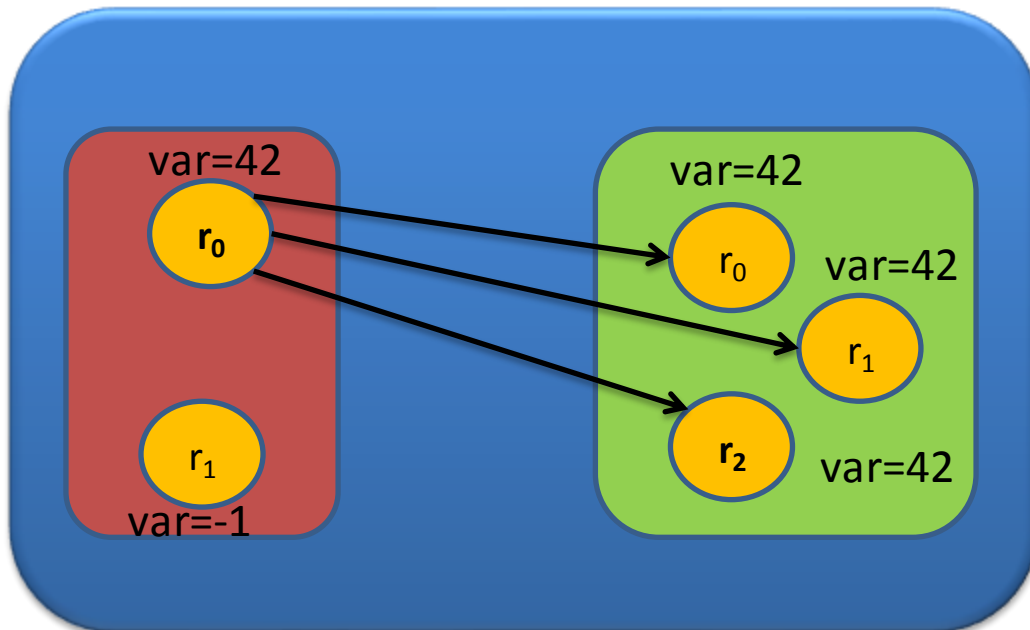
intracomm



Bcast intercommunicator

- A Bcast on an **intercommunicator**, the root “sends” its data to all other MPI processes in the **OTHER** group of the communicator
 - The MPI processes in the same group won't have the data

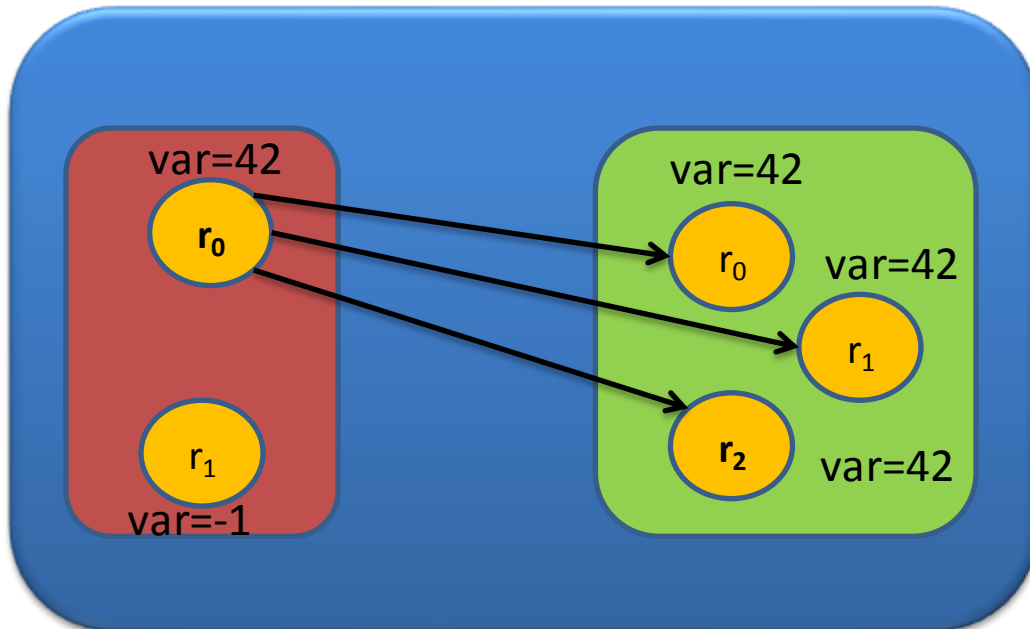
new_intercomm



Rooted collective on intercomm

- How to specify the root of the Bcast (or any rooted collective)

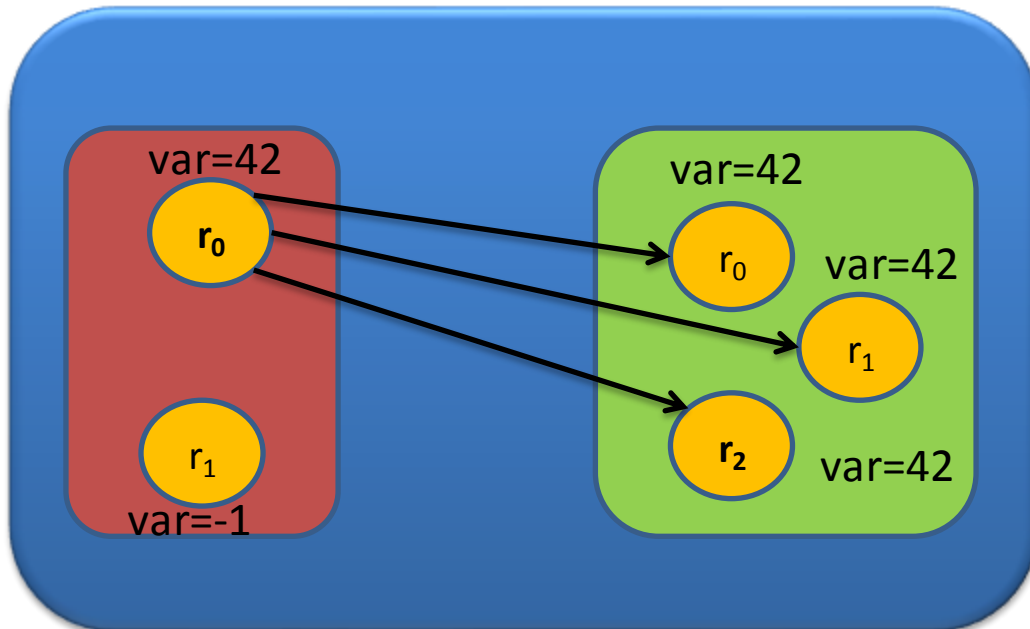
new_intercomm



Rooted collective on intercomm

- How to specify the root of the Bcast (or any rooted collective)
 - In my example, root has rank 0...
 - ... but I have two MPI processes with rank 0 in the intercommunicator !

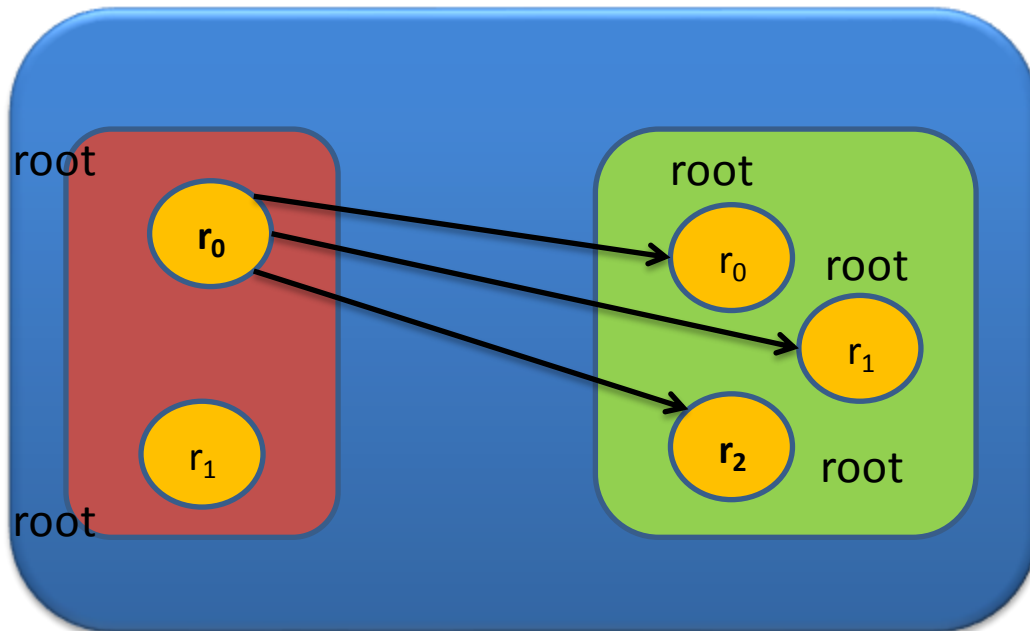
new_intercomm



Rooted collective on intercomm

- Solution: MPI processes involved in the rooted collective on an intercommunicator will give different root arguments:

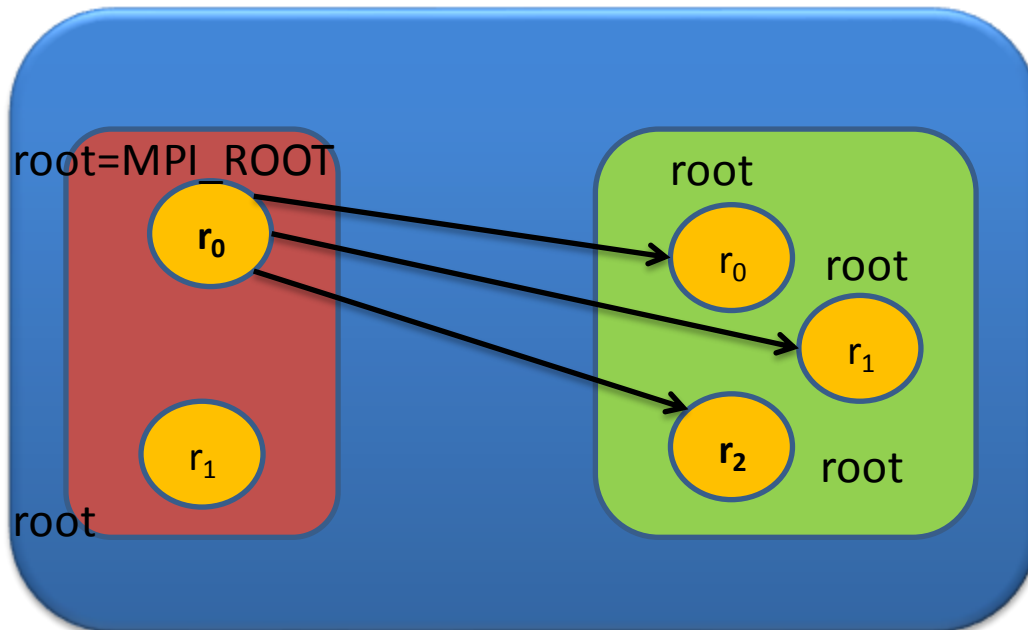
new_intercomm



Rooted collective on intercomm

- Solution: MPI processes involved in the rooted collective on an intercommunicator will give different root arguments:
 - The root MPI process gives the predefined value `MPI_ROOT`

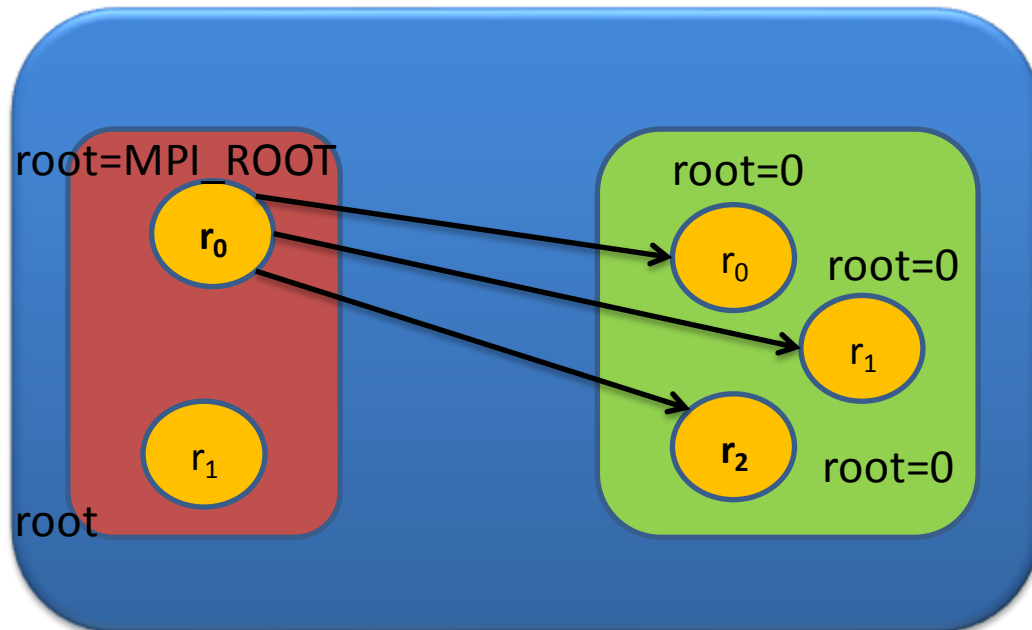
`new_intercomm`



Rooted collective on intercomm

- Solution: MPI processes involved in the rooted collective on an intercommunicator will give different root arguments:
 - The root MPI process gives the predefined value `MPI_ROOT`

`new_intercomm`

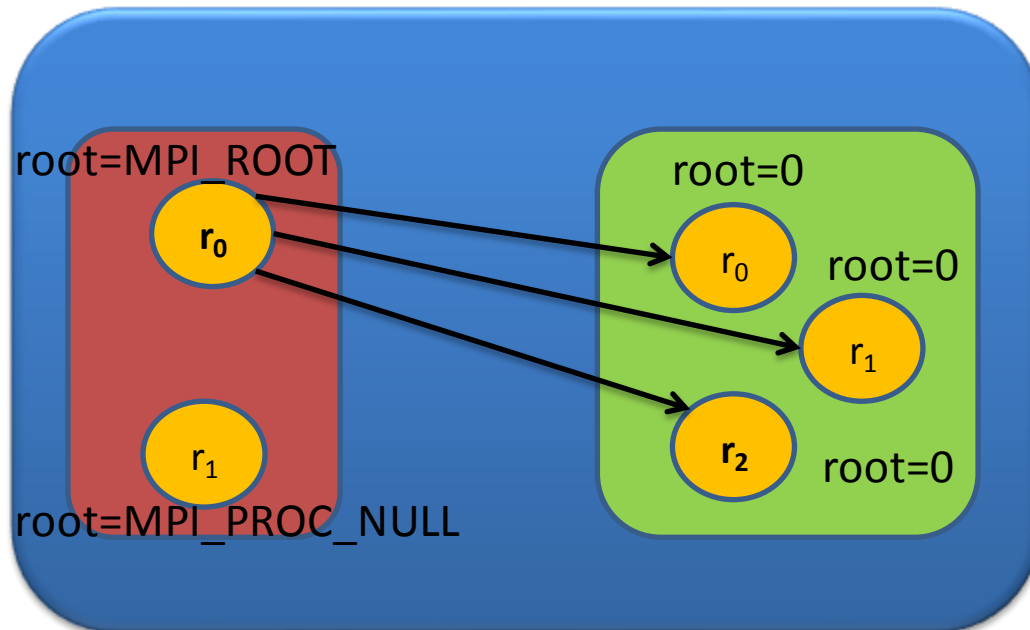


- The receiver MPI processes (hence MPI processes in the other group than the root) gives the rank of the root in its own group
 - In this example : 0

Rooted collective on intercomm

- Solution: MPI processes involved in the rooted collective on an intercommunicator will give different root arguments:
 - The root MPI process gives the predefined value `MPI_ROOT`

new_intercomm



- The receiver MPI processes (hence MPI processes in the other group than the root) gives the rank of the root in its own group
 - In this example : 0
- The other MPI processes in the same group of the root MPI process gives the predefined value `MPI_PROC_NULL`

Bcast intercomm example

■ Example code

```
MPI_Comm newcomm;  
MPI_Comm_split(MPI_COMM_WORLD, worldrank%2, worldrank, &newcomm);
```

```
MPI_Comm intercomm;  
MPI_Intercomm_create(newcomm, 0, MPI_COMM_WORLD, (worldrank+1)%2, 1000, &intercomm);
```

```
if(worldrank %2 == root%2)  
{  
    if(worldrank == root)  
    {  
        MPI_Bcast(&val, 1, MPI_INT, MPI_ROOT, intercomm);  
    }  
    else  
    {  
        MPI_Bcast(&val, 1, MPI_INT, MPI_PROC_NULL, intercomm);  
    }  
}  
else  
{  
    MPI_Bcast(&val, 1, MPI_INT, interroot, intercomm);  
}
```

Bcast intercomm example

```
MPI_Comm newcomm;  
MPI_Comm_split(MPI_COMM_WORLD, worldrank%2, worldrank, &newcomm);
```

```
MPI_Comm intercomm;  
MPI_Intercomm_create(newcomm, 0, MPI_COMM_WORLD, (worldrank+1)%2, 1000, &intercomm);
```

```
if(worldrank %2 == root%2)  
{  
    if(worldrank == root)  
    {  
        MPI_Bcast(&val, 1, MPI_INT, MPI_ROOT, intercomm);  
    }  
    else  
    {  
        MPI_Bcast(&val, 1, MPI_INT, MPI_PROC_NULL, intercomm);  
    }  
}  
else  
{  
    MPI_Bcast(&val, 1, MPI_INT, interroot, intercomm);  
}
```




Bcast intercomm example



```
MPI_Comm newcomm;  
MPI_Comm_split(MPI_COMM_WORLD, worldrank%2, worldrank, &newcomm);
```

```
MPI_Comm intercomm;  
MPI_Intercomm_create(newcomm, 0, MPI_COMM_WORLD, (worldrank+1)%2, 1000, &intercomm);
```

```
MPI_Bcast(&interroot, 1, MPI_INT, root, MPI_COMM_WORLD);
```

```
if(worldrank %2 == root%2)  
{  
    if(worldrank == root)  
    {  
        MPI_Bcast(&val, 1, MPI_INT, MPI_ROOT, intercomm);  
    }  
    else  
    {  
        MPI_Bcast(&val, 1, MPI_INT, MPI_PROC_NULL, intercomm);  
    }  
}  
else  
{  
    MPI_Bcast(&val, 1, MPI_INT, interroot, intercomm);  
}
```

Bcast intercomm example

- Example output

```
BEFORE the bcast I am 0 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is 8
BEFORE the bcast I am 1 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is 8
BEFORE the bcast I am 2 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is 8
BEFORE the bcast I am 3 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is 8
BEFORE the bcast I am 4 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is 8
BEFORE the bcast I am 5 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is 8
BEFORE the bcast I am 6 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is 8
BEFORE the bcast I am 7 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is 8
BEFORE the bcast I am 8 in WORLD, and 4 in NEW_comm, and 4 in intercomm, and my val is 8
I am 3 in world, the root of the broadcast, and my val is 42
AFTER the bcast I am 0 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is 42
AFTER the bcast I am 1 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is 8
AFTER the bcast I am 2 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is 42
AFTER the bcast I am 3 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is 42
AFTER the bcast I am 4 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is 42
AFTER the bcast I am 5 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is 8
AFTER the bcast I am 6 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is 42
AFTER the bcast I am 7 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is 8
AFTER the bcast I am 8 in WORLD, and 4 in NEW_comm, and 4 in intercomm, and my val is 42
```

Bcast intercomm example

- Example output

```
BEFORE the bcast I am 0 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is 8
BEFORE the bcast I am 1 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is 8
BEFORE the bcast I am 2 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is 8
BEFORE the bcast I am 3 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is 8
BEFORE the bcast I am 4 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is 8
BEFORE the bcast I am 5 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is 8
BEFORE the bcast I am 6 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is 8
BEFORE the bcast I am 7 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is 8
BEFORE the bcast I am 8 in WORLD, and 4 in NEW_comm, and 4 in intercomm, and my val is 8
I am 3 in world, the root of the broadcast, and my val is 42
AFTER the bcast I am 0 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is 42
AFTER the bcast I am 1 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is 8
AFTER the bcast I am 2 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is 42
AFTER the bcast I am 3 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is 42
AFTER the bcast I am 4 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is 42
AFTER the bcast I am 5 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is 8
AFTER the bcast I am 6 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is 42
AFTER the bcast I am 7 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is 8
AFTER the bcast I am 8 in WORLD, and 4 in NEW_comm, and 4 in intercomm, and my val is 42
```



Scatter intercomm example



```
MPI_Comm newcomm;  
MPI_Comm_split(MPI_COMM_WORLD, worldrank%2, worldrank, &newcomm);
```

```
MPI_Comm intercomm;  
MPI_Intercomm_create(newcomm, 0, MPI_COMM_WORLD, (worldrank+1)%2, 1000, &intercomm);
```

```
MPI_Bcast(&interroot, 1, MPI_INT, root, MPI_COMM_WORLD);
```

```
if(worldrank %2 == root %2)  
{  
    if(worldrank == root)  
    {  
        MPI_Scatter(&sendbuff, 1, MPI_INT, &val, 1, MPI_INT, MPI_ROOT, intercomm);  
    }  
    else  
    {  
        MPI_Scatter(NULL, 1, MPI_INT, &val, 1, MPI_INT, MPI_PROC_NULL, intercomm);  
    }  
}  
else  
{  
    MPI_Scatter(NULL, 1, MPI_INT, &val, 1, MPI_INT, interroot, intercomm);  
}
```

Scatter intercomm example

■ Example output

```
BEFORE the bcast I am 0 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is -1
BEFORE the bcast I am 1 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is -1
BEFORE the bcast I am 2 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is -1
BEFORE the bcast I am 3 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is -1
BEFORE the bcast I am 4 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is -1
BEFORE the bcast I am 5 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is -1
BEFORE the bcast I am 6 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is -1
BEFORE the bcast I am 7 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is -1
BEFORE the bcast I am 8 in WORLD, and 4 in NEW_comm, and 4 in intercomm, and my val is -1
I am 3 in world, the root of the broadcast, and my sendbuff is: [383][886][777][915][793][335][386][492][649]
AFTER the bcast I am 0 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is 383
AFTER the bcast I am 1 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is -1
AFTER the bcast I am 2 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is 886
AFTER the bcast I am 3 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is -1
AFTER the bcast I am 4 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is 777
AFTER the bcast I am 5 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is -1
AFTER the bcast I am 6 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is 915
AFTER the bcast I am 7 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is -1
AFTER the bcast I am 8 in WORLD, and 4 in NEW_comm, and 4 in intercomm, and my val is 793
```

Scatter intercomm example

- Example output

```
BEFORE the bcast I am 0 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is -1
BEFORE the bcast I am 1 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is -1
BEFORE the bcast I am 2 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is -1
BEFORE the bcast I am 3 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is -1
BEFORE the bcast I am 4 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is -1
BEFORE the bcast I am 5 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is -1
BEFORE the bcast I am 6 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is -1
BEFORE the bcast I am 7 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is -1
BEFORE the bcast I am 8 in WORLD, and 4 in NEW_comm, and 4 in intercomm, and my val is -1
I am 3 in world, the root of the broadcast, and my sendbuff is: [383][886][777][915][793][335][386][492][649]
AFTER the bcast I am 0 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is 383
AFTER the bcast I am 1 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my val is -1
AFTER the bcast I am 2 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is 886
AFTER the bcast I am 3 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my val is -1
AFTER the bcast I am 4 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is 777
AFTER the bcast I am 5 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my val is -1
AFTER the bcast I am 6 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is 915
AFTER the bcast I am 7 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my val is -1
AFTER the bcast I am 8 in WORLD, and 4 in NEW_comm, and 4 in intercomm, and my val is 793
```


Gather intercomm example

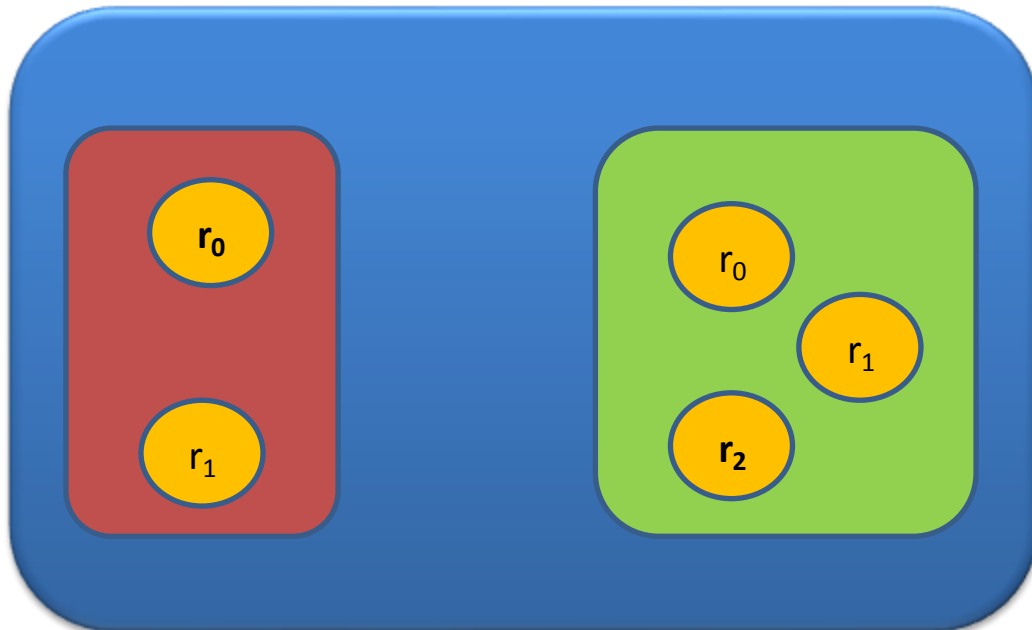
- Example output

```
I am 0 in WORLD and val is myrank (i.e., 0)
I am 1 in WORLD and val is myrank (i.e., 1)
I am 2 in WORLD and val is myrank (i.e., 2)
I am 3 in WORLD and val is myrank (i.e., 3)
I am 4 in WORLD and val is myrank (i.e., 4)
I am 5 in WORLD and val is myrank (i.e., 5)
I am 6 in WORLD and val is myrank (i.e., 6)
I am 7 in WORLD and val is myrank (i.e., 7)
I am 8 in WORLD and val is myrank (i.e., 8)
I am 3 in world, the root of the gather, and before the gather, my recv buffer is
[-1][-1][-1][-1][-1][-1][-1][-1][-1]
I am 3 in world, the root of the gather, and AFTER the gather, my recv buffer is
[0][2][4][6][8][-1][-1][-1][-1]
```

Non-rooted collective on intercomm.

- What happens for non-rooted collective ?
 - E.g. All-to-all collectives: MPI_Alltoall, MPI_Allgather, MPI_Allreduce, MPI_Reduce_scatter and their derivatives

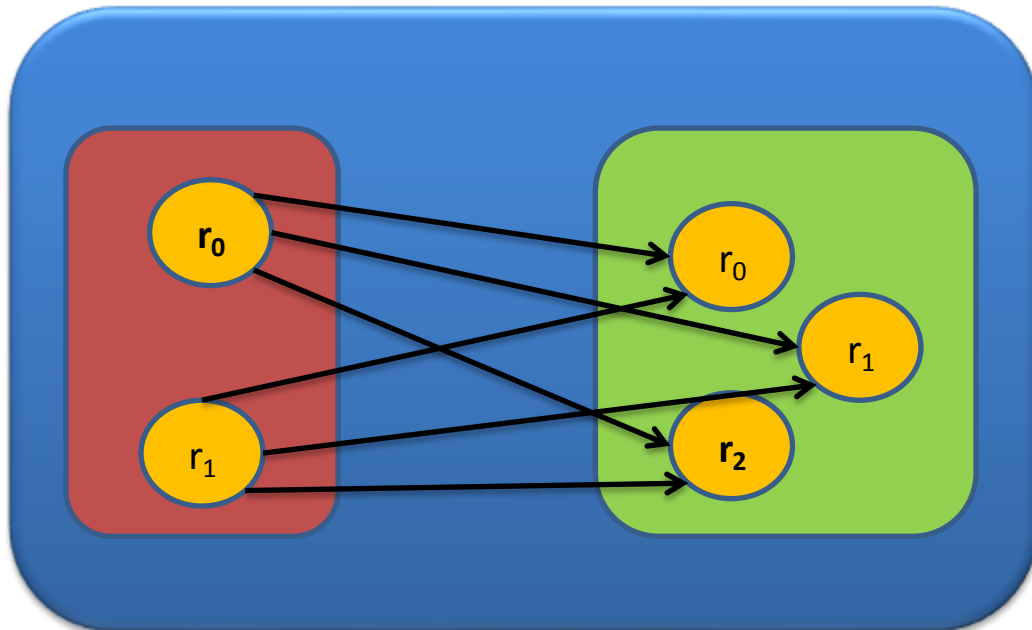
new_intercomm



Non-rooted collective on intercomm.

- What happens for non-rooted collective
 - MPI processes of each groups only send and receive data from all MPI processes of the other group

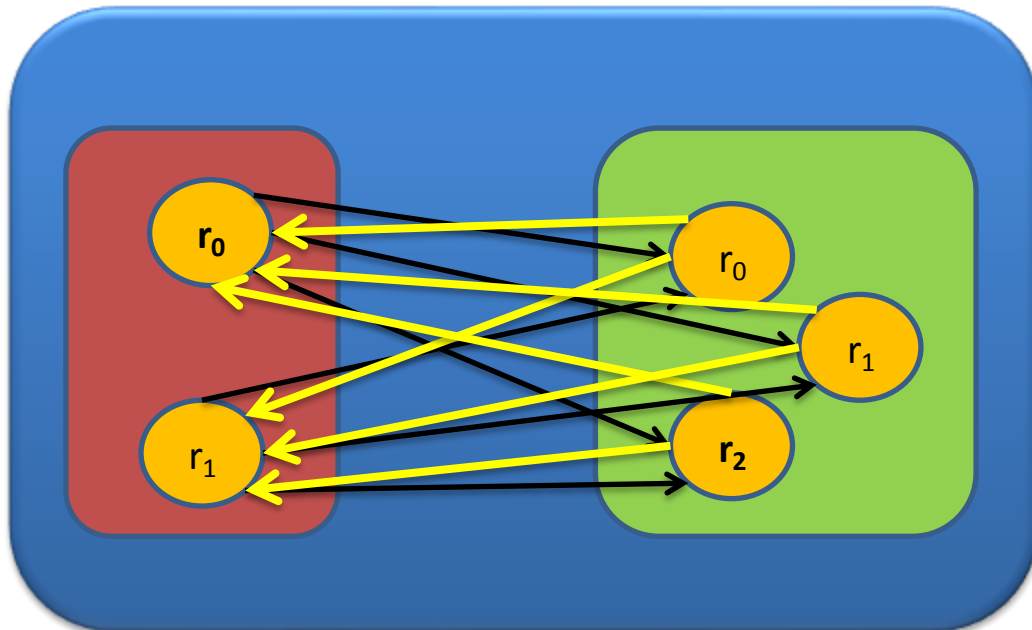
new_intercomm



Non-rooted collective on intercomm.

- What happens for non-rooted collective
 - MPI processes of each groups only send and receive data from all MPI processes of the other group

new_intercomm



Allreduce intercomm example



- Much more simple to call
 - No root = same call for every MPI processes

```
MPI_Comm newcomm;  
MPI_Comm_split(MPI_COMM_WORLD, worldrank%2, worldrank, &newcomm);  
MPI_Comm_rank(newcomm, &rank);
```

```
MPI_Comm intercomm;  
MPI_Intercomm_create(newcomm, 0, MPI_COMM_WORLD, (worldrank+1)%2, 1000, &intercomm);
```

```
MPI_Allreduce(&val, &redval, 1, MPI_INT, MPI_SUM, intercomm);
```

Allreduce intercomm example

- Example output

```
I am 0 in WORLD and val is my rank (i.e., 0)
I am 1 in WORLD and val is my rank (i.e., 1)
I am 2 in WORLD and val is my rank (i.e., 2)
I am 3 in WORLD and val is my rank (i.e., 3)
I am 4 in WORLD and val is my rank (i.e., 4)
I am 5 in WORLD and val is my rank (i.e., 5)
I am 6 in WORLD and val is my rank (i.e., 6)
I am 7 in WORLD and val is my rank (i.e., 7)
I am 8 in WORLD and val is my rank (i.e., 8)
I am 0 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my redval is 16
I am 1 in WORLD, and 0 in NEW_comm, and 0 in intercomm, and my redval is 20
I am 2 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my redval is 16
I am 3 in WORLD, and 1 in NEW_comm, and 1 in intercomm, and my redval is 20
I am 4 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my redval is 16
I am 5 in WORLD, and 2 in NEW_comm, and 2 in intercomm, and my redval is 20
I am 6 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my redval is 16
I am 7 in WORLD, and 3 in NEW_comm, and 3 in intercomm, and my redval is 20
I am 8 in WORLD, and 4 in NEW_comm, and 4 in intercomm, and my redval is 16
```

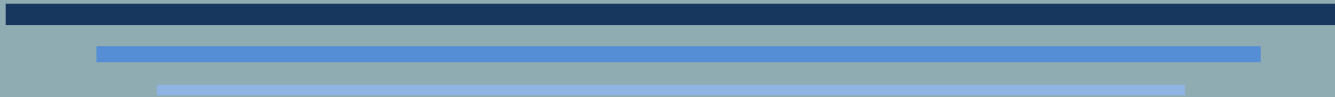
Allgather intercomm example

■ Example output

```
I am 0 in WORLD and val is myrank (i.e., 0)
I am 1 in WORLD and val is myrank (i.e., 1)
I am 2 in WORLD and val is myrank (i.e., 2)
I am 3 in WORLD and val is myrank (i.e., 3)
I am 4 in WORLD and val is myrank (i.e., 4)
I am 5 in WORLD and val is myrank (i.e., 5)
I am 6 in WORLD and val is myrank (i.e., 6)
I am 7 in WORLD and val is myrank (i.e., 7)
I am 8 in WORLD and val is myrank (i.e., 8)

I am 0 in world, and AFTER the allgather, my recv buffer is
[1][3][5][7][-1][-1][-1][-1][-1]
I am 1 in world, and AFTER the allgather, my recv buffer is
[0][2][4][6][8][-1][-1][-1][-1]
I am 2 in world, and AFTER the allgather, my recv buffer is
[1][3][5][7][-1][-1][-1][-1][-1]
I am 3 in world, and AFTER the allgather, my recv buffer is
[0][2][4][6][8][-1][-1][-1][-1]
I am 4 in world, and AFTER the allgather, my recv buffer is
[1][3][5][7][-1][-1][-1][-1][-1]
I am 5 in world, and AFTER the allgather, my recv buffer is
[0][2][4][6][8][-1][-1][-1][-1]
I am 6 in world, and AFTER the allgather, my recv buffer is
[1][3][5][7][-1][-1][-1][-1][-1]
I am 7 in world, and AFTER the allgather, my recv buffer is
[0][2][4][6][8][-1][-1][-1][-1]
I am 8 in world, and AFTER the allgather, my recv buffer is
[1][3][5][7][-1][-1][-1][-1][-1]
[jaegerj@login2 allgather] $
```

MPI TOPOLOGY



MPI Topology



- MPI processes are arbitrarily distributed on resources, with no regards to the communications patterns in play
- Simply because the MPI runtime does not have any clue of the communications that will happen on a communicator
- The user is responsible for the communication patterns, which reflect the data repartition across the MPI processes

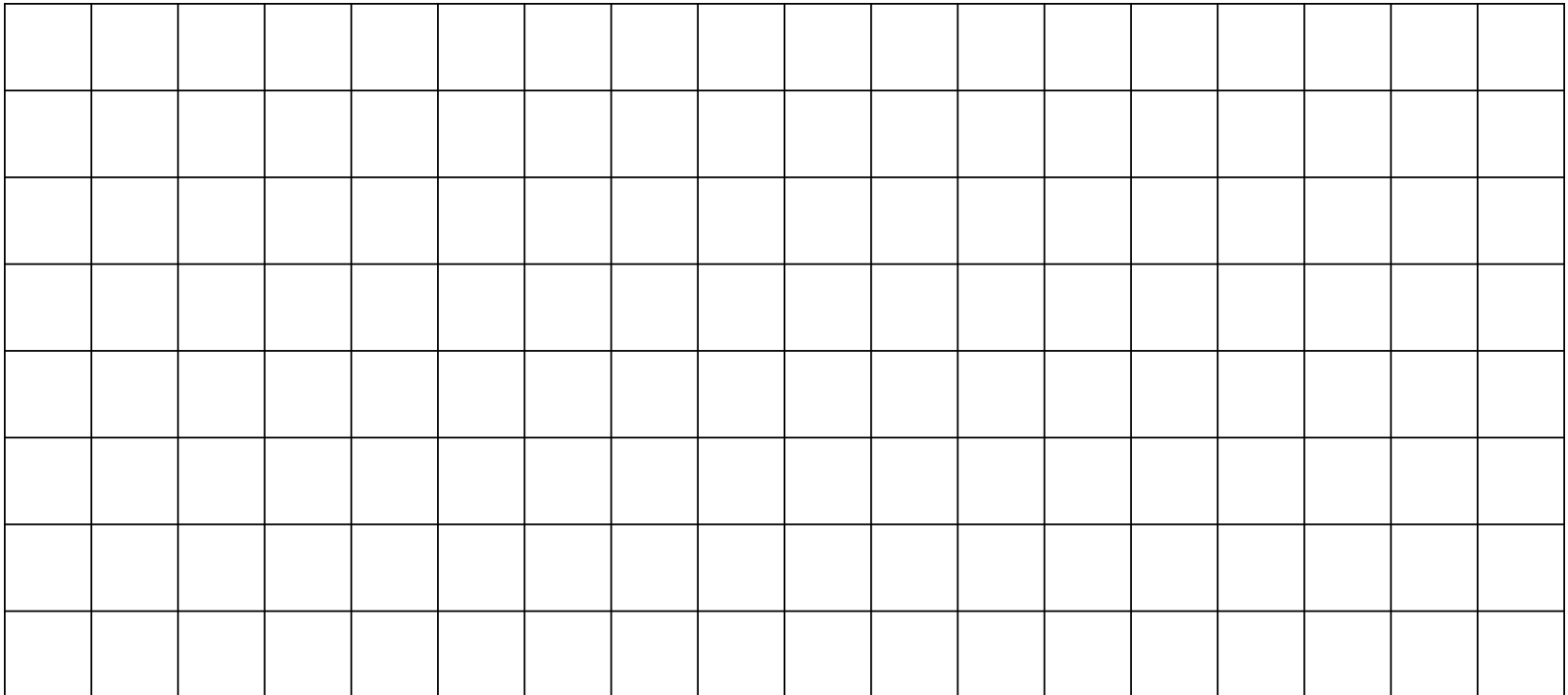
MPI Topology

- Because it is easier for humans, data repartition often follows constructed patterns
- MPI Topology routines allow to virtually organize MPI processes according to a given topology
- Two types of topology necessary: Multi-dimensional grids or graphs

MPI Grid Topology

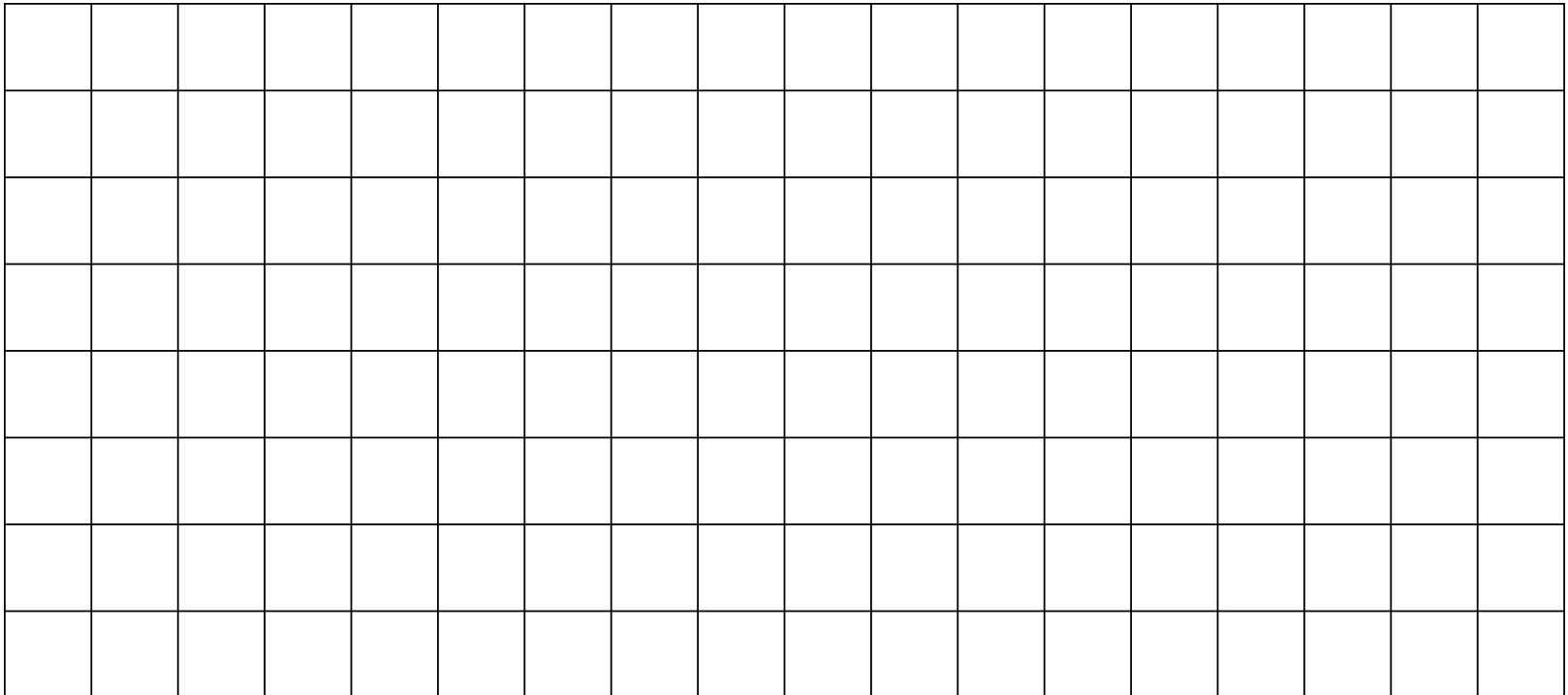


- Consider a multi-dimensional grid
 - 2D in the example



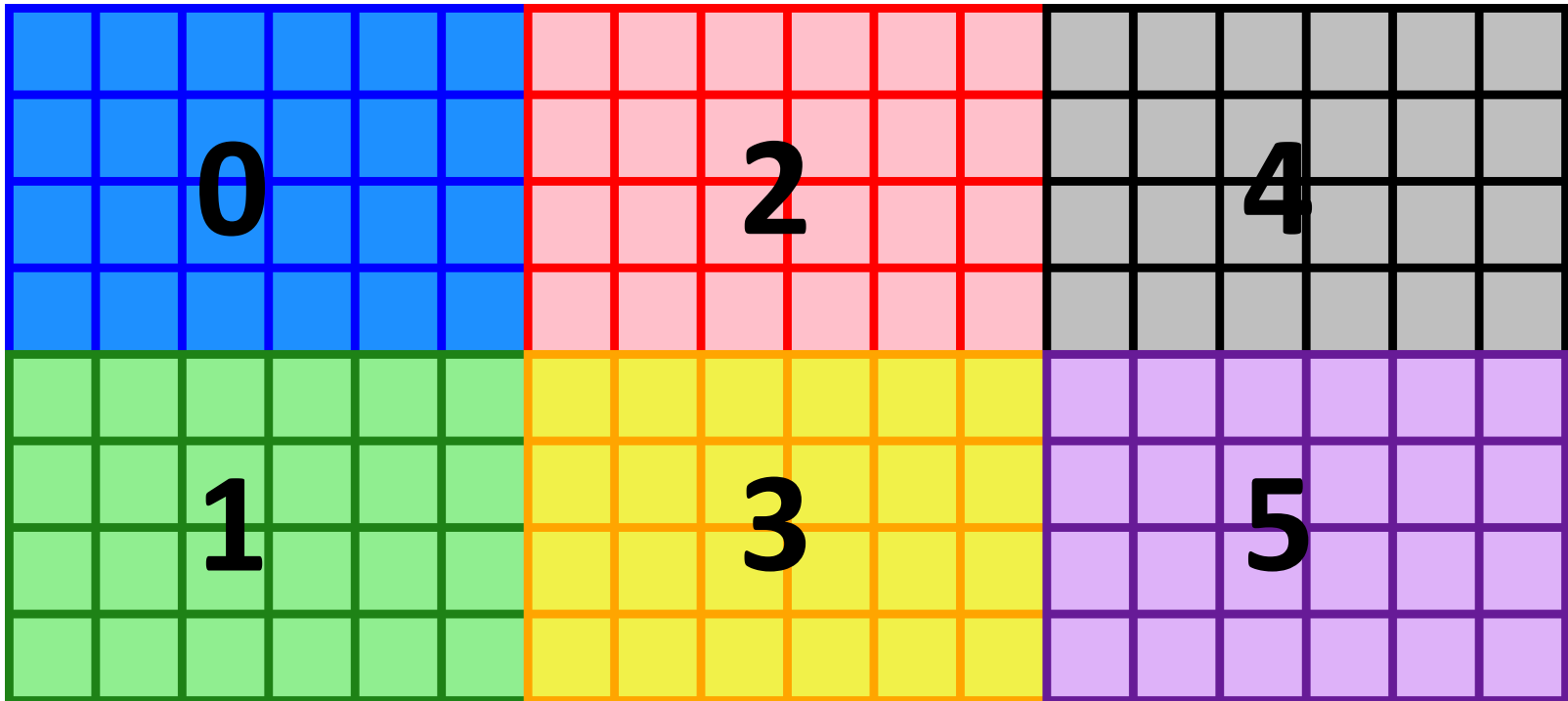
MPI Grid Topology

- Homogeneous cells
 - Homogeneous loads
- Decomposing this grid on 6 MPI ranks



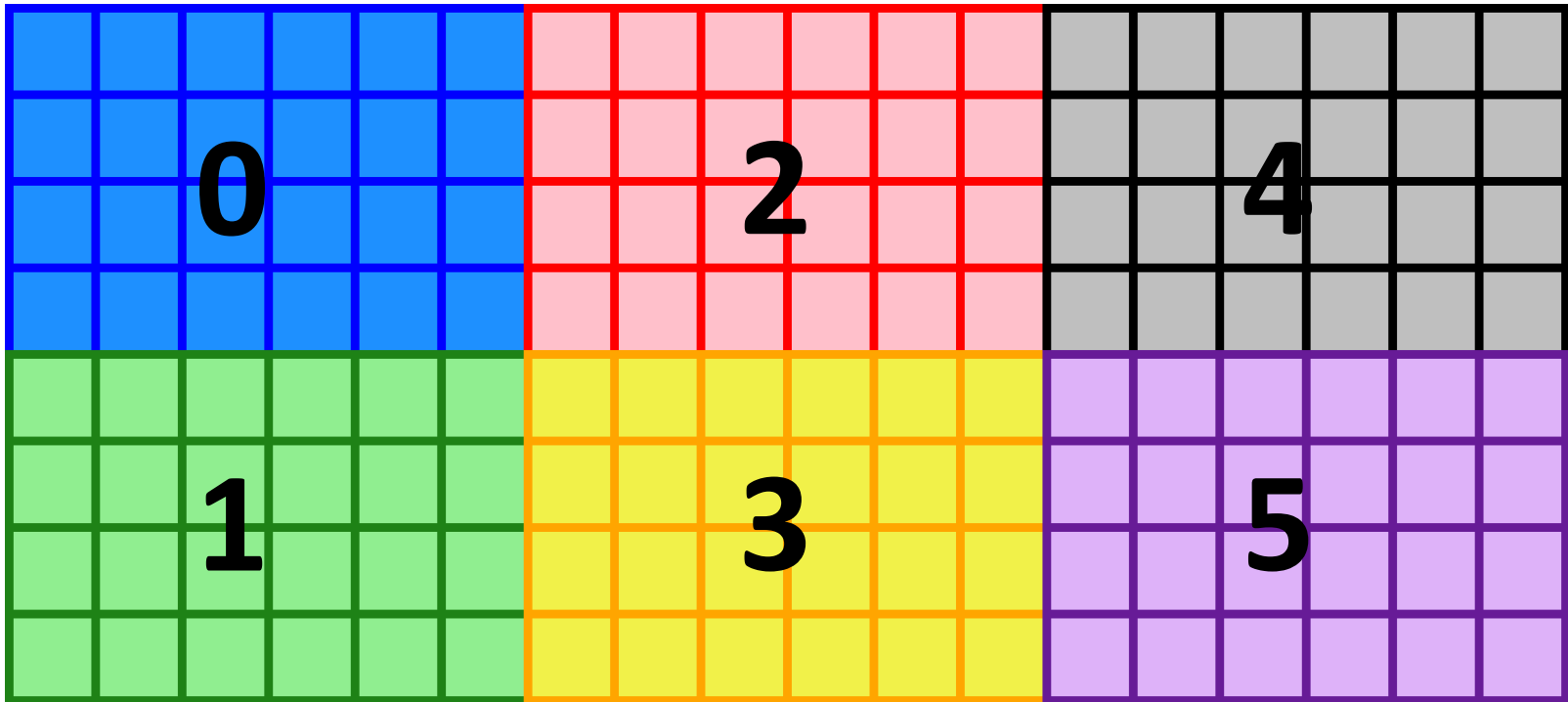
MPI Grid Topology

- Homogeneous decomposition
 - $6 \times 4 = 24$ cells per MPI rank



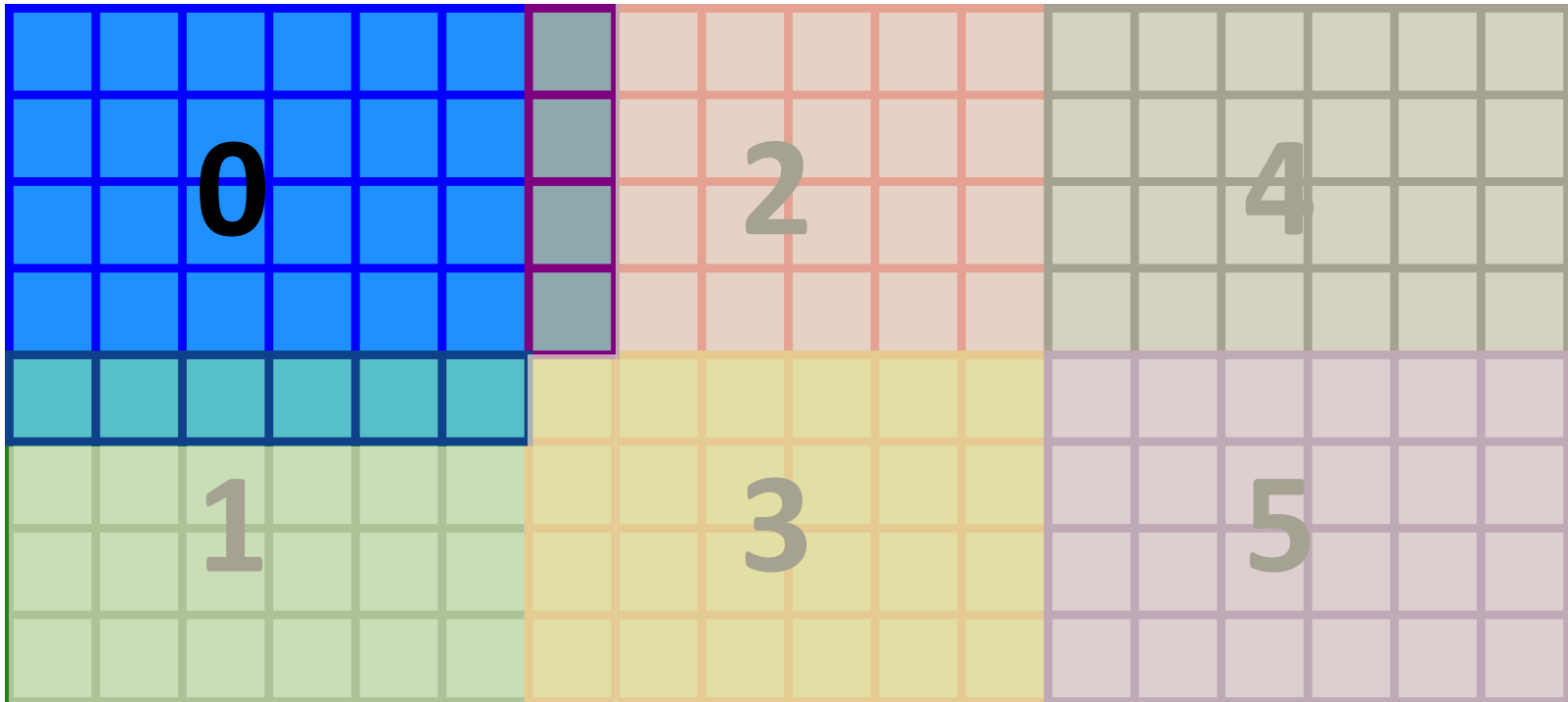
MPI Grid Topology

- Stencil code: updating a cell requires contribution from direct neighbors



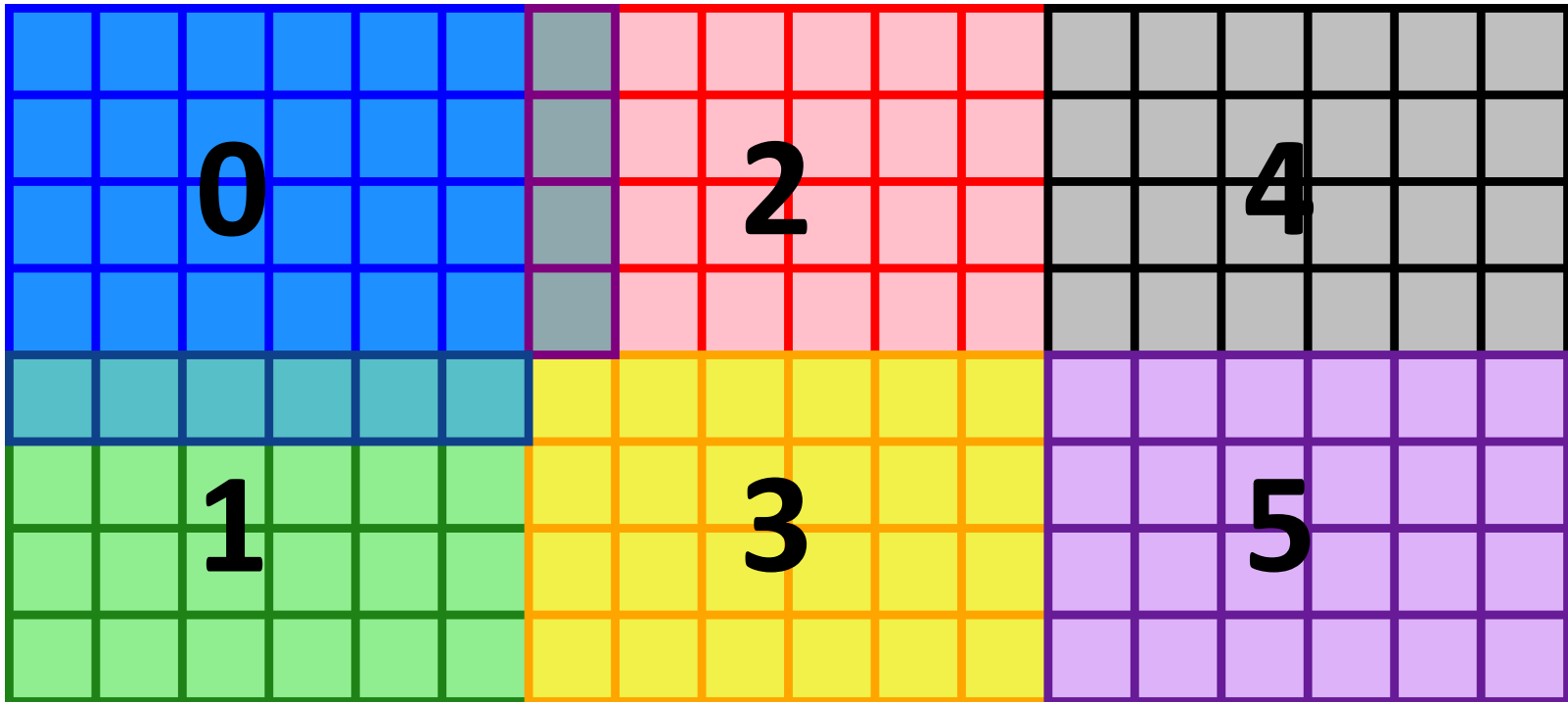
MPI Grid Topology

- Stencil code: updating a cell requires contribution from direct neighbors
 - Highlighted cell for MPI rank 0



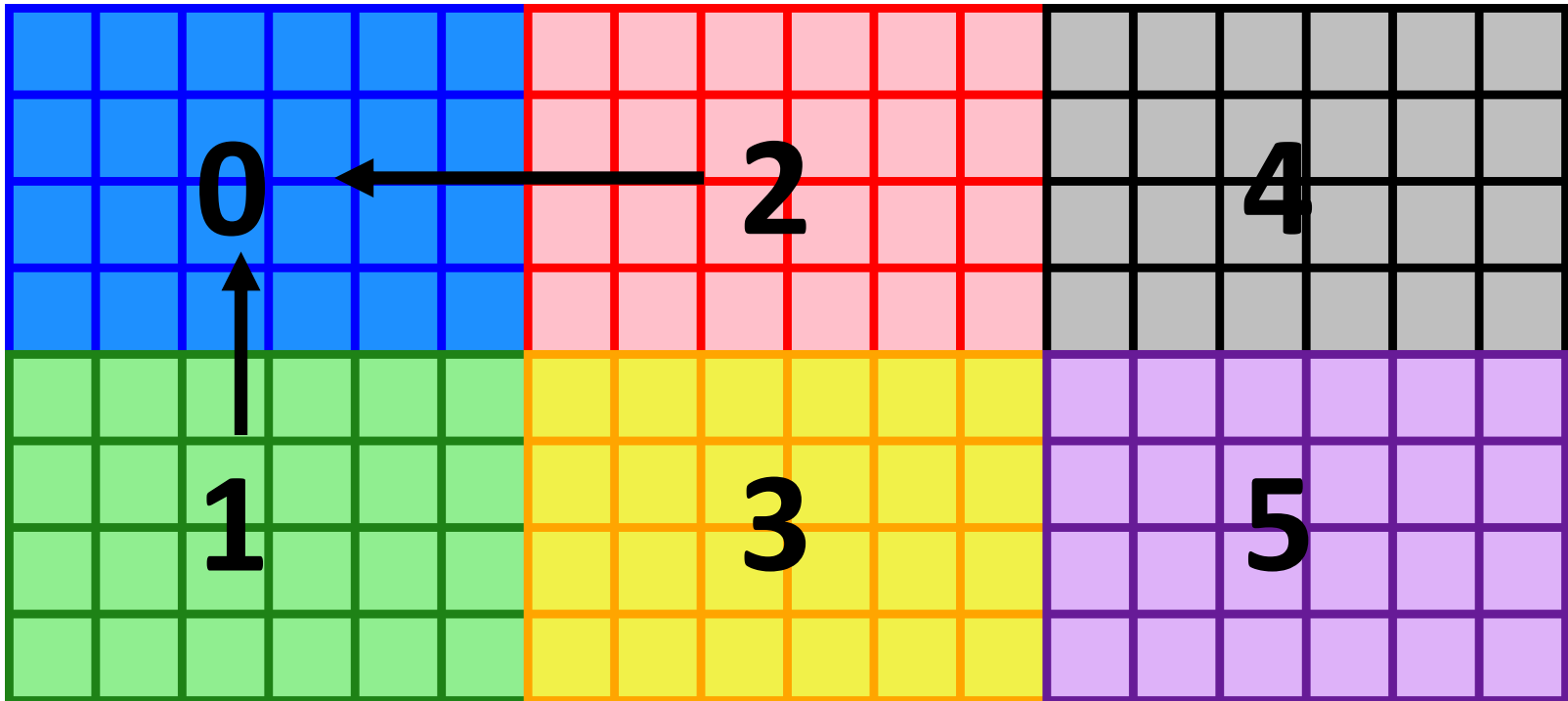
MPI Grid Topology

- Stencil code: updating a cell requires contribution from direct neighbors
 - Highlighted cell for MPI rank 0



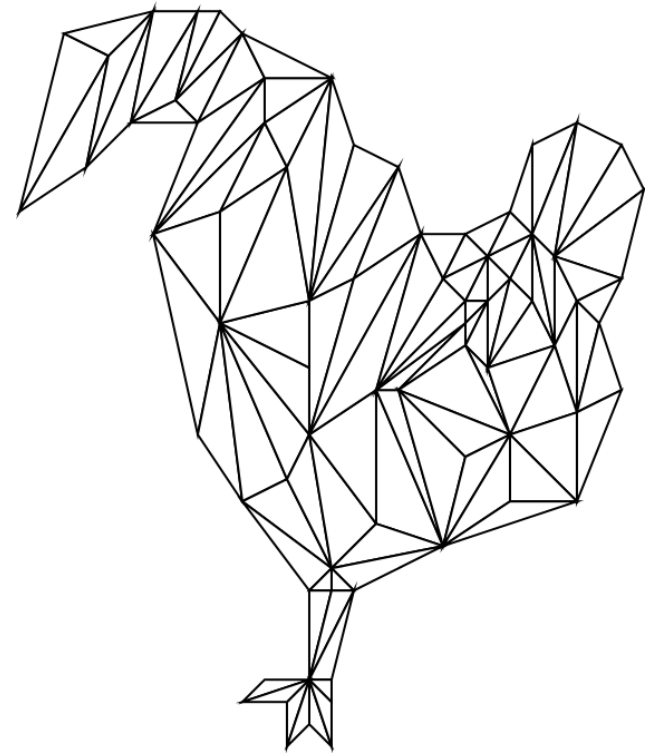
MPI Grid Topology

- A rank needs to exchange data only with its neighbors
 - Use a ***cart*** topology to fix rank neighbors in MPI runtime



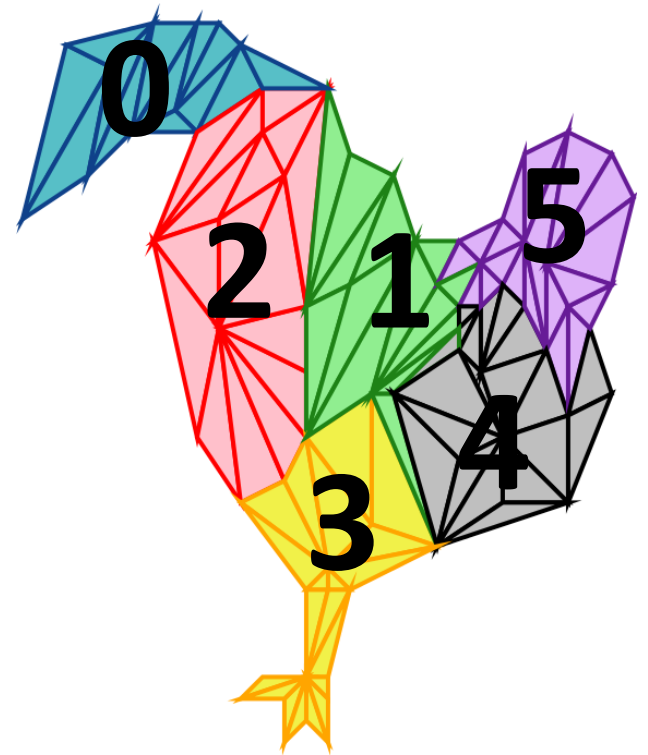
MPI Graph Topology

- Consider a non-uniform mesh representing an irregular form
 - 2D in the example
- Usually, call an automatic practitioner to decompose data across MPI ranks
 - With different metrics: number of cells/particles/comms, amount of comms...



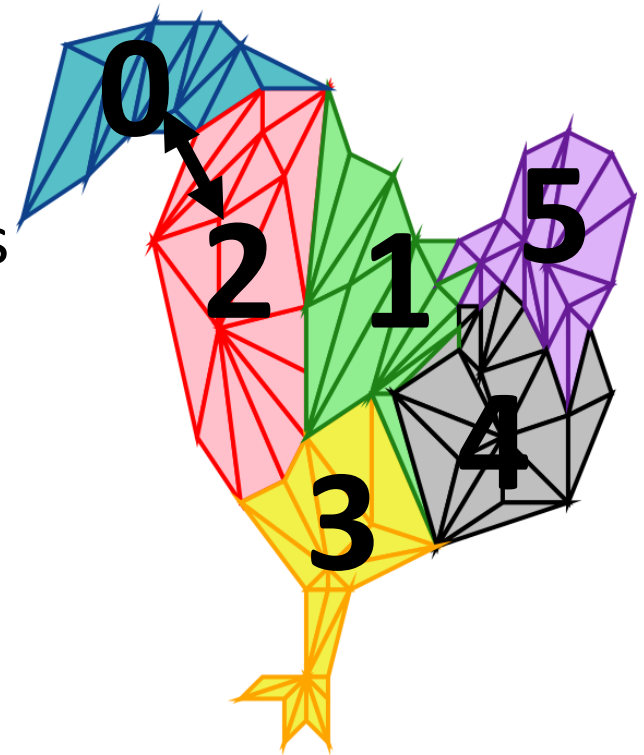
MPI Graph Topology

- Decomposing this mesh on 6 MPI ranks



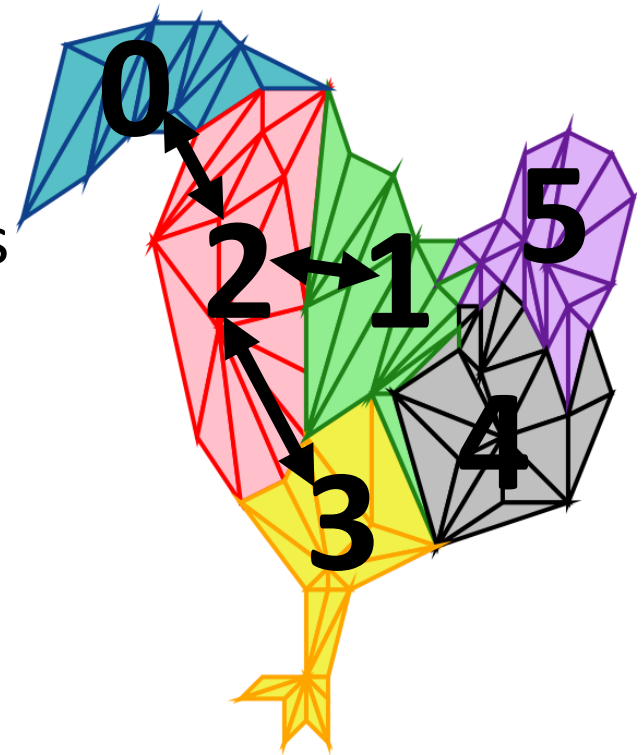
MPI Graph Topology

- Decomposing this mesh on 6 MPI ranks
- Particles movement: A rank needs to exchange data only with its neighbors
 - $0 \leftrightarrow 2$



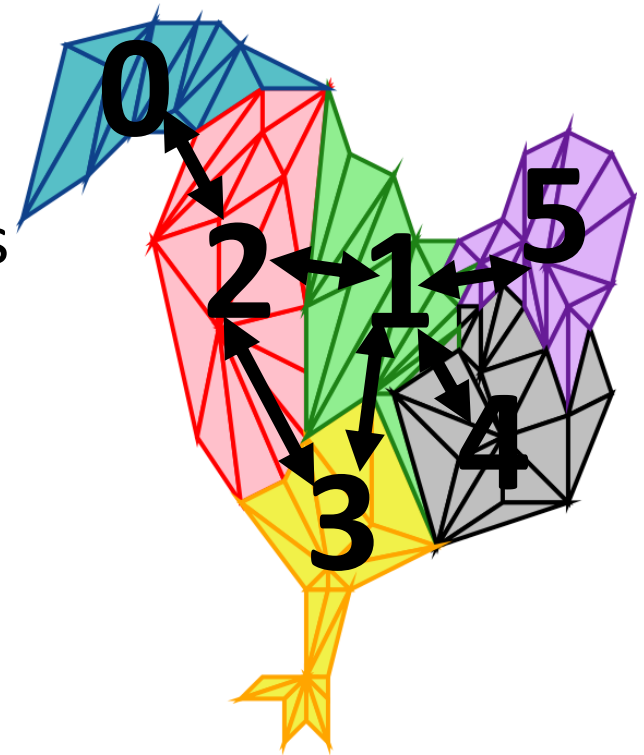
MPI Graph Topology

- Decomposing this mesh on 6 MPI ranks
- Particles movement: A rank needs to exchange data only with its neighbors
 - $0 \leftrightarrow 2$
 - $2 \leftrightarrow 1$; $2 \leftrightarrow 3$



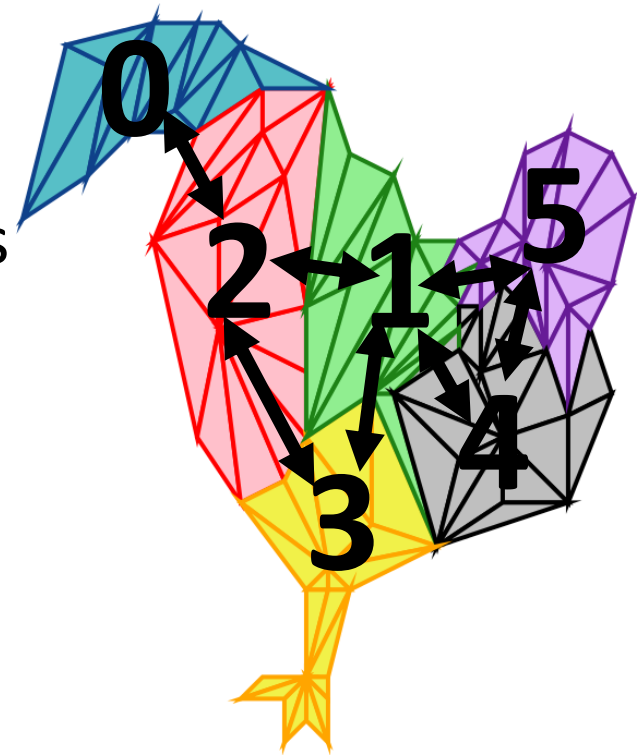
MPI Graph Topology

- Decomposing this mesh on 6 MPI ranks
- Particles movement: A rank needs to exchange data only with its neighbors
 - $0 \leftrightarrow 2$
 - $2 \leftrightarrow 1$; $2 \leftrightarrow 3$
 - $1 \leftrightarrow 3$; $1 \leftrightarrow 4$; $1 \leftrightarrow 5$



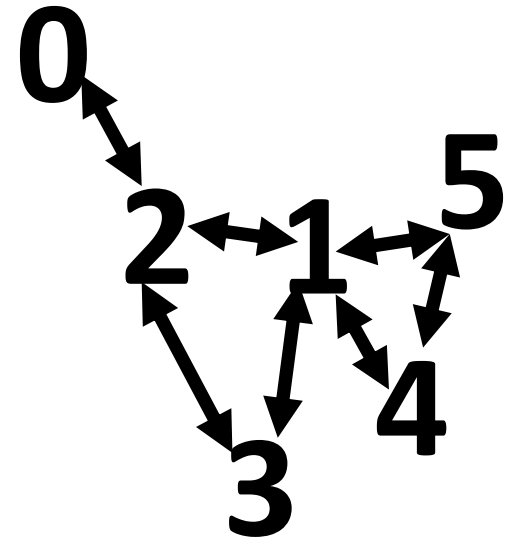
MPI Graph Topology

- Decomposing this mesh on 6 MPI ranks
- Particles movement: A rank needs to exchange data only with its neighbors
 - $0 \leftrightarrow 2$
 - $2 \leftrightarrow 1$; $2 \leftrightarrow 3$
 - $1 \leftrightarrow 3$; $1 \leftrightarrow 4$; $1 \leftrightarrow 5$
 - $4 \leftrightarrow 5$;



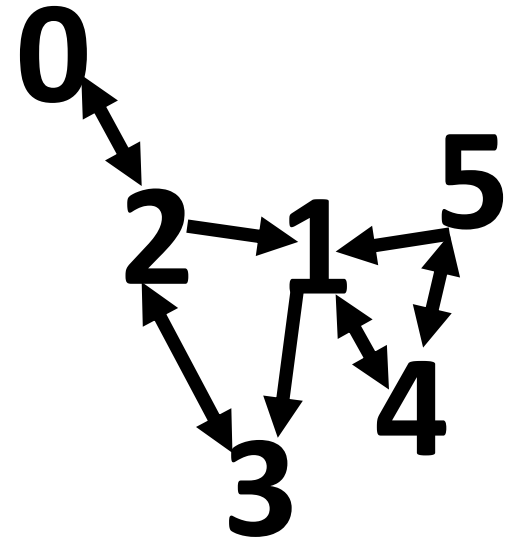
MPI Graph Topology

- You get a graph of communications between MPI ranks
- Using a **graph** topology allow to specify this graph to the MPI runtime
- Collectives communications will only involve neighbors according to the topology



MPI Graph Topology

- Need to specify neighbors per MPI ranks
 - Possible to have unidirectional vertices in the graph
- Graph creation exists in two forms:
 - Each MPI Rank must specify the complete topology
 - Not scalable
 - Each MPI Rank only specifies its own neighbors



Cartesian Topology creation

- `MPI_Cart_create(MPI_Comm comm_old, int ndims, const int dims[], const int periods[], int reorder, MPI_Comm *comm_cart);`
 - IN `comm_old` (input communicator)
 - IN `ndims` (number of dimensions of Cartesian grid)
 - IN `dims` (integer array of size `ndims` specifying the number of processes in each dimension)
 - IN `periods` (logical array of size `ndims` specifying whether the grid is periodic *-true-* or not *-false-* in each dim.)
 - IN `reorder` (ranking may be logically reordered *-true-* or not *-false-*)
 - OUT `comm_cart` (communicator with new Cartesian topology)

MPI_Cart_create example 1



```
int size_array=2;  
int dims[2] = {3 , 2};  
int periods[2] = {false, false};  
MPI_Cart_create(comm_old, size_array, dims, periods, false, &comm_cart);
```

■ Size_array = 2 => 2D array ↘

MPI_Cart_create example 1

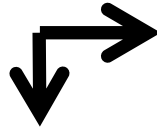
```
int size_array=2;
```

```
int dims[2] = {3 , 2};
```

```
int periods[2] = {false, false};
```

```
MPI_Cart_create(comm_old, size_array, dims, periods, false, &comm_cart);
```

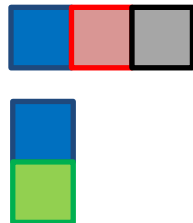
- Size_array = 2 => 2D array
- dims[0] = 3



MPI_Cart_create example 1

```
int size_array=2;  
int dims[2] = {3 , 2};  
int periods[2] = {false, false};  
MPI_Cart_create(comm_old, size_array, dims, periods, false, &comm_cart);
```

- Size_array = 2 => 2D array
- dims[0] = 3
- dims[1] = 2



MPI_Cart_create example 1

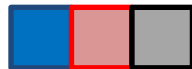
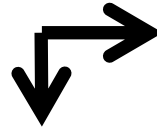
```
int size_array=2;  
int dims[2] = {3 , 2};  
int periods[2] = {false, false};  
MPI_Cart_create(comm_old, size_array, dims, periods, false, &comm_cart);
```

■ Size_array = 2 => 2D array

■ dims[0] = 3

■ dims[1] = 2

■ topology



MPI_Cart_create example 1

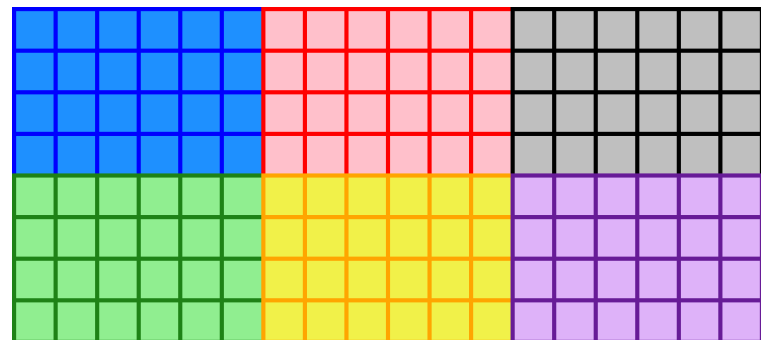
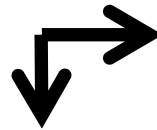
```
int size_array=2;  
int dims[2] = {3 , 2};  
int periods[2] = {false, false};  
MPI_Cart_create(comm_old, size_array, dims, periods, false, &comm_cart);
```

■ Size_array = 2 => 2D array

■ dims[0] = 3

■ dims[1] = 2

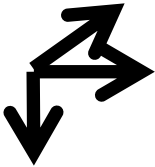
■ topology



MPI_Cart_create example 2



```
int size_array=3;  
int dims[3] = {3 , 2, 4};  
int periods[2] = {true , false, false};  
MPI_Cart_create(comm_old, size_array, dims, periods, false, &comm_cart);
```

■ Size_array = 3 => 3D array 

MPI_Cart_create example 2

```
int size_array=3;
```

```
int dims[3] = {3 , 2, 4};
```

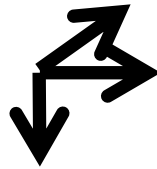
```
int periods[2] = {true , false, false};
```

```
MPI_Cart_create(comm_old, size_array, dims, periods, false, &comm_cart);
```

- Size_array = 3 => 3D array

- dims[0] = 3

--	--	--

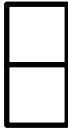


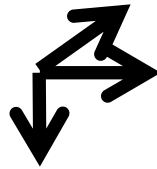
MPI_Cart_create example 2

```
int size_array=3;  
int dims[3] = {3 , 2, 4};  
int periods[2] = {true , false, false};  
MPI_Cart_create(comm_old, size_array, dims, periods, false, &comm_cart);
```

■ Size_array = 3 => 3D array

■ dims[0] = 3 

■ dims[1] = 2 



MPI_Cart_create example 2

```
int size_array=3;  
int dims[3] = {3 , 2, 4};  
int periods[2] = {true , false, false};  
MPI_Cart_create(comm_old, size_array, dims, periods, false, &comm_cart);
```

■ Size_array = 3 => 3D array

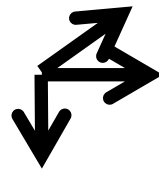
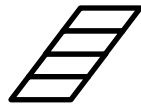
■ dims[0] = 3



■ dims[1] = 2



■ dims[3] = 4



MPI_Cart_create example 2

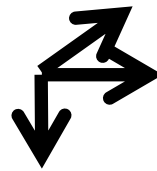
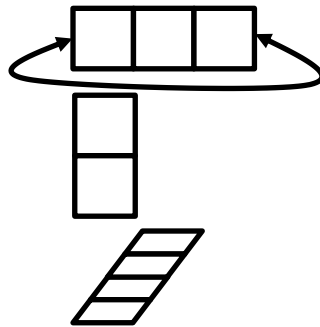
```
int size_array=3;  
int dims[3] = {3 , 2, 4};  
int periods[2] = {true , false, false};  
MPI_Cart_create(comm_old, size_array, dims, periods, false, &comm_cart);
```

■ Size_array = 3 => 3D array

■ dims[0] = 3

■ dims[1] = 2

■ dims[3] = 4



periods[0] = true

MPI_Cart_create example 2

```
int size_array=3;  
int dims[3] = {3 , 2, 4};  
int periods[2] = {true , false, false};  
MPI_Cart_create(comm_old, size_array, dims, periods, false, &comm_cart);
```

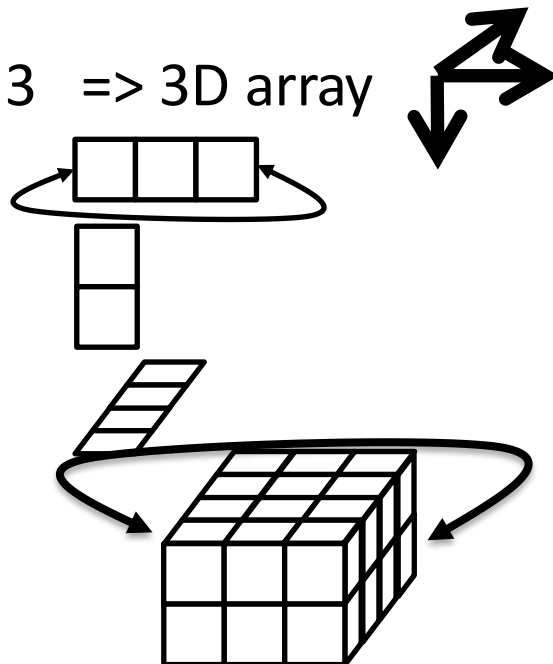
- Size_array = 3 => 3D array

- dims[0] = 3

- dims[1] = 2

- dims[3] = 4

- topology:



periods[0] = true

Get MPI rank coordinate

- `MPI_Cart_coords(MPI_Comm comm, int rank, int maxdims, int coords[]);`
 - IN `comm` (communicator with Cartesian structure)
 - IN `rank` (rank of a process within group of `comm`)
 - IN `maxdims` (length of `coords` - *ndims* - in the calling program)
 - OUT `coords` (integer array (of size *ndims*) containing the Cartesian coordinates of specified rank)
- Returns, in array *coords*, the coordinates of the specified *rank* in the cartesian topology associated with communicator *comm*.

Get MPI rank from coordinates



- `MPI_Cart_rank`(MPI_Comm comm, const int coords[], int *rank);
 - IN comm (communicator with Cartesian structure)
 - IN coords (integer array (of size *ndims*) specifying the Cartesian coordinates of a process)
 - OUT rank (rank of specified process)
- Returns, in *rank*, the rank located at the coordinates *coords* in the cartesian topology associated with communicator *comm*.

Get Cartesian Topology dimensions



- `MPI_Cartdim_get(MPI_Comm comm, int *ndims);`
 - IN `comm` (communicator with Cartesian structure)
 - OUT `ndims` (number of dimensions of the Cartesian structure)
- Returns, in *ndims*, the number of dimensions in the cartesian topology associated with communicator *comm*.

Get Cartesian Topology infos



- `MPI_Cart_get(MPI_Comm comm, int maxdims, int dims[], int periods[], int coords[]);`
 - IN comm (communicator with Cartesian structure)
 - IN maxdims (length of vectors dims, periods, and coords)
 - OUT dims (number of processes for each Cartesian dimension)
 - OUT periods (periodicity *-true/false-* for each Cartesian dimension)
 - OUT coords (coordinates of calling process in Cartesian structure)
- Returns the number of dimension, and the periodicity of these dimensions, of the cartesian topology associated with communicator *comm*. Also returns the coordinates of the calling *rank* in the cartesian topology

Automatically create topo dimensions

- 
- 
- `MPI_Dims_create(nnodes, ndims, dims);`
 - IN `nnodes` (number of nodes in a grid)
 - IN `ndims` (number of Cartesian dimensions)
 - INOUT `dims` (integer array of size `ndims` specifying the number of nodes in each dimension)
 - This function will compute the size of each dimensions according to
 - The number of MPI ranks for the cartesian topology
 - The number of dimensions in the future topology
 - A first partial repartition specified by the user
 - If `dims[i]` is not 0 on input, the value is kept
 - If `dims[i]` is 0 on input, the function replace it with the computed value for dimension `i`
 - The dimensions are as close to each other as possible

Automatically create topo dimensions

- Examples from the MPI API document
 - Chapter 8.5.2, page 394 MPI 4.0

DIMS BEFORE CALL	FUNCTION CALL	DIMS ON RETURN
(0,0)	MPI_DIMS_CREATE(6, 2, dims)	(3,2)
(0,0)	MPI_DIMS_CREATE(7, 2, dims)	(7,1)
(0,3,0)	MPI_DIMS_CREATE(6, 3, dims)	(2,3,1)
(0,3,0)	MPI_DIMS_CREATE(7, 3, dims)	erroneous call

Automatically create topo dimensions

- Examples from the MPI API document
 - Chapter 7.5, page 293

DIMS BEFORE CALL	FUNCTION CALL	DIMS ON RETURN
(0,0)	MPI_DIMS_CREATE(6, 2, dims)	(3,2)
(0,0)	MPI_DIMS_CREATE(7, 2, dims)	(7,1)
(0,3,0)	MPI_DIMS_CREATE(6, 3, dims)	(2,3,1)
(0,3,0)	MPI_DIMS_CREATE(7, 3, dims)	erroneous call

- 6 ranks, 2 dimensions, no predefined sizes
 - $6 = 6 \times 1 = \mathbf{3 \times 2} = 2 \times 3 = 1 \times 6$

Automatically create topo dimensions

- Examples from the MPI API document
 - Chapter 7.5, page 293

DIMS BEFORE CALL	FUNCTION CALL	DIMS ON RETURN
(0,0)	MPI_DIMS_CREATE(6, 2, dims)	(3,2)
(0,0)	MPI_DIMS_CREATE(7, 2, dims)	(7,1)
(0,3,0)	MPI_DIMS_CREATE(6, 3, dims)	(2,3,1)
(0,3,0)	MPI_DIMS_CREATE(7, 3, dims)	erroneous call

- 7 ranks, 2 dimensions, no predefined sizes
 - $7 = \mathbf{7 \times 1} = 1 \times 7$

Automatically create topo dimensions

- Examples from the MPI API document
 - Chapter 7.5, page 293

DIMS BEFORE CALL	FUNCTION CALL	DIMS ON RETURN
(0,0)	MPI_DIMS_CREATE(6, 2, dims)	(3,2)
(0,0)	MPI_DIMS_CREATE(7, 2, dims)	(7,1)
(0,3,0)	MPI_DIMS_CREATE(6, 3, dims)	(2,3,1)
(0,3,0)	MPI_DIMS_CREATE(7, 3, dims)	erroneous call

- 6 ranks, 3 dimensions, 1 predefined dimension
 - $6 = \mathbf{2 \times 3 \times 1} = 1 \times \underline{3} \times 2 = \cancel{3 \times 2 \times 1} = \dots = \cancel{6 \times 1 \times 1} = \cancel{1 \times 6 \times 1} = \dots$

Automatically create topo dimensions

- Examples from the MPI API document
 - Chapter 7.5, page 293

DIMS BEFORE CALL	FUNCTION CALL	DIMS ON RETURN
(0,0)	MPI_DIMS_CREATE(6, 2, dims)	(3,2)
(0,0)	MPI_DIMS_CREATE(7, 2, dims)	(7,1)
(0,3,0)	MPI_DIMS_CREATE(6, 3, dims)	(2,3,1)
(0,3,0)	MPI_DIMS_CREATE(7, 3, dims)	erroneous call

- 7 ranks, 3 dimensions, 1 predefined dimension
 - $7 = \cancel{7 \times 1 \times 1} = \cancel{1 \times 7 \times 1} = \cancel{1 \times 1 \times 7}$



Graph Topology creation



- `MPI_Graph_create(MPI_Comm comm_old, int nnodes, const int index[], const int edges[], int reorder, MPI_Comm *comm_graph);`
 - IN `comm_old` (input communicator)
 - IN `nnodes` (number of nodes in graph)
 - IN `index` (array of integers describing node degrees)
 - IN `edges` (array of integers describing graph edges)
 - IN `reorder` (ranking may be reordered *-true-* or *-false-*)
 - OUT `comm_graph` (communicator with graph topology added)
- Index: cumulative number of neighbors
 - The *i*-th entry of array *index* stores the total number of neighbors of the first *i* graph nodes.
- Edges: linearized array of edges indexes of all nodes (ordered)

MPI_Graph_create example

- What you want:

process	neighbors
0	1,3
1	0,3
2	
3	0,2

```
int nnodes=4;
```

```
int index[4] = {2 , 4, 4, 6};
```

```
int edges[index[nnodes-1]] = {1, 3, 0, 3, 0, 2};
```

```
MPI_Graph_create(comm_old, nnodes, index, edges, false, &comm_graph);
```

- nnodes =4

0 2
1 3

MPI_Graph_create example

- What you want:

process	neighbors
0	1,3
1	0,3
2	
3	0,2

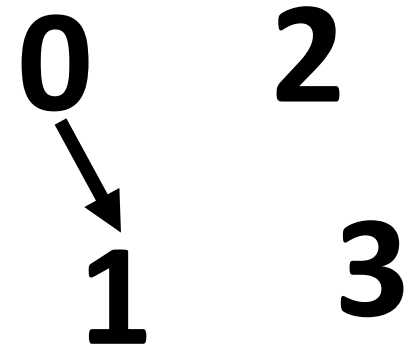
```
int nnodes=4;
```

```
int index[4] = {2, 4, 4, 6};
```

```
int edges[index[nnodes-1]] = {1, 3, 0, 3, 0, 2};
```

```
MPI_Graph_create(comm_old, nnodes, index, edges, false, &comm_graph);
```

- `nnodes = 4`
- `index[0] = 2`
 - `edge[0]=1`



MPI_Graph_create example

- What you want:

process	neighbors
0	1,3
1	0,3
2	
3	0,2

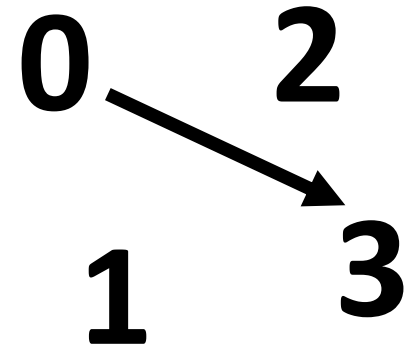
```
int nnodes=4;
```

```
int index[4] = {2, 4, 4, 6};
```

```
int edges[index[nnodes-1]] = {1, 3, 0, 3, 0, 2};
```

```
MPI_Graph_create(comm_old, nnodes, index, edges, false, &comm_graph);
```

- `nnodes = 4`
- `index[0] = 2`
 - `edge[0]=1`
 - `edge[1]=3`



MPI_Graph_create example

- What you want:

process	neighbors
0	1,3
1	0,3
2	
3	0,2

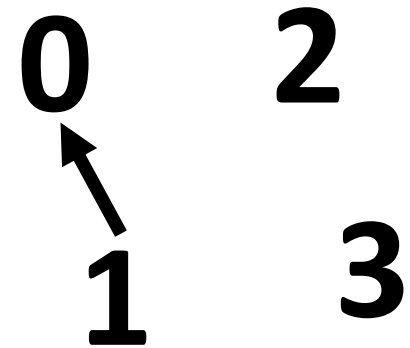
```
int nnodes=4;
```

```
int index[4] = {2, 4, 4, 6};
```

```
int edges[index[nnodes-1]] = {1, 3, 0, 3, 0, 2};
```

```
MPI_Graph_create(comm_old, nnodes, index, edges, false, &comm_graph);
```

- `nnodes = 4`
- `index[0] = 2`
 - `edge[0]=1`
 - `edge[1]=3`
- `Index[1]-index[0] = 2`
 - `edge[2] = 0`



MPI_Graph_create example

- What you want:

process	neighbors
0	1,3
1	0,3
2	
3	0,2

```
int nnodes=4;
```

```
int index[4] = {2, 4, 4, 6};
```

```
int edges[index[nnodes-1]] = {1, 3, 0, 3, 0, 2};
```

```
MPI_Graph_create(comm_old, nnodes, index, edges, false, &comm_graph);
```

- `nnodes = 4`

- `index[0] = 2`

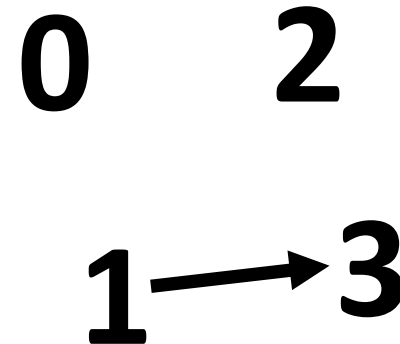
- `edge[0]=1`

- `edge[1]=3`

- `Index[1]-index[0] = 2`

- `edge[2] = 0`

- `edge[3]=3`



MPI_Graph_create example

- What you want:

process	neighbors
0	1,3
1	0,3
2	
3	0,2

```
int nnodes=4;
```

```
int index[4] = {2 , 4, 4, 6};
```

```
int edges[index[nnodes-1]] = {1, 3, 0, 3, 0, 2};
```

```
MPI_Graph_create(comm_old, nnodes, index, edges, false, &comm_graph);
```

- $nnodes = 4$

- $index[0] = 2$

- $edge[0]=1$

- $edge[1]=3$

- $Index[1]-index[0] = 2$

- $edge[2] = 0$

- $edge[3]=3$

- $index[2]-index[1] = 0$

0

2

1

3

MPI_Graph_create example

- What you want:

process	neighbors
0	1,3
1	0,3
2	
3	0,2

```
int nnodes=4;
```

```
int index[4] = {2, 4, 4, 6};
```

```
int edges[index[nnodes-1]] = {1, 3, 0, 3, 0, 2};
```

```
MPI_Graph_create(comm_old, nnodes, index, edges, false, &comm_graph);
```

- $nnodes = 4$

- $index[0] = 2$

- $edge[0] = 1$

- $edge[1] = 3$

- $Index[1] - index[0] = 2$

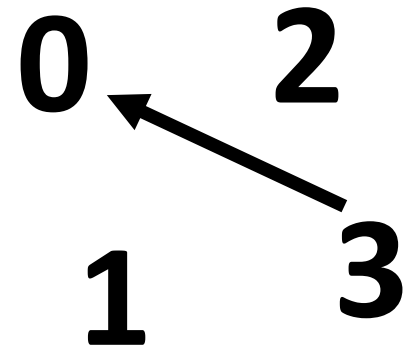
- $edge[2] = 0$

- $edge[3] = 3$

- $index[2] - index[1] = 0$

- $Index[3] - index[2] = 2$

- $edge[4] = 0$



MPI_Graph_create example

- What you want:

process	neighbors
0	1,3
1	0,3
2	
3	0,2

```
int nnodes=4;
```

```
int index[4] = {2, 4, 4, 6};
```

```
int edges[index[nnodes-1]] = {1, 3, 0, 3, 0, 2};
```

```
MPI_Graph_create(comm_old, nnodes, index, edges, false, &comm_graph);
```

- $nnodes = 4$

- $index[0] = 2$

- $edge[0] = 1$

- $edge[1] = 3$

- $Index[1] - index[0] = 2$

- $edge[2] = 0$

- $edge[3] = 3$

- $index[2] - index[1] = 0$

- $Index[3] - index[2] = 2$

- $edge[4] = 0$

- $edge[5] = 2$

0

1

2

3

MPI_Graph_create example

- What you want:

process	neighbors
0	1,3
1	0,3
2	
3	0,2

```
int nnodes=4;
```

```
int index[4] = {2, 4, 4, 6};
```

```
int edges[index[nnodes-1]] = {1, 3, 0, 3, 0, 2};
```

```
MPI_Graph_create(comm_old, nnodes, index, edges, false, &comm_graph);
```

- $nnodes = 4$

- $index[0] = 2$

- $edge[0] = 1$

- $edge[1] = 3$

- $Index[1] - index[0] = 2$

- $edge[2] = 0$

- $edge[3] = 3$

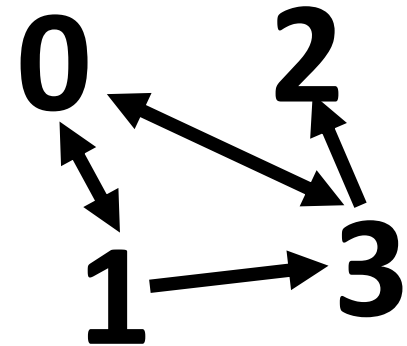
- $index[2] - index[1] = 0$

- $Index[3] - index[2] = 2$

- $edge[4] = 0$

- $edge[5] = 2$

- Topology:



Neighborhood Graph Topology creat.



```
int MPI_Dist_graph_create_adjacent (  
    MPI_Comm comm_old(in),  
    int indegree(in),  
    const int sources[](in),  
    const int sourceweights[](in),  
    int outdegree (in),  
    const int destinations[](in),  
    const int destweights[] (in),  
    MPI_Info info(in),  
    int reorder(in),  
    MPI_Comm *comm_dist_graph (out)  
);
```

`comm_old` is the initial
communicator containing the processes
we want in the topology

Neighborhood Graph Topology creat.



```
int MPI_Dist_graph_create_adjacent (
```

```
    MPI_Comm comm_old(in),
```

```
    int indegree(in),
```

```
    const int sources[](in),
```

```
    const int sourceweights[](in),
```

```
    int outdegree (in),
```

```
    const int destinations[](in),
```

```
    const int destweights[] (in),
```

```
    MPI_Info info(in),
```

```
    int reorder(in),
```

```
    MPI_Comm *comm_dist_graph (out)
```

```
);
```

`indegree`: number of incoming edges
= number of processes with current rank as destination.

`sources[indegree]`: list of processes
with current rank as a destination.

`sourceweights[indegree]`: weight of
each incoming edges.

Neighborhood Graph Topology creat.



```
int MPI_Dist_graph_create_adjacent (  
    MPI_Comm comm_old(in),  
    int indegree(in),  
    const int sources[](in),  
    const int sourceweights[](in),  
    int outdegree(in),  
    const int destinations[](in),  
    const int destweights[](in),  
    MPI_Info info(in),  
    int reorder(in),  
    MPI_Comm *comm_dist_graph(out)  
);
```

`outdegree`: number of exiting edges =
number of processes with current rank as a
source.

`destinations[outdegree]`: list of
processes with current rank as a source.

`Sourceweights[outdegree]`: weight
of each exiting edges.

Neighborhood Graph Topology creat.



```
int MPI_Dist_graph_create_adjacent (  
    MPI_Comm comm_old(in),  
    int indegree(in),  
    const int sources[](in),  
    const int sourceweights[](in),  
    int outdegree (in),  
    const int destinations[](in),  
    const int destweights[] (in),  
    MPI_Info info(in),  
    int reorder(in),  
    MPI_Comm *comm_dist_graph (out)  
);
```

`info` may be used to influence interpretation of weights (higher is better or lower is better, ...)

Neighborhood Graph Topology creat.



```
int MPI_Dist_graph_create_adjacent (  
    MPI_Comm comm_old(in),  
    int indegree(in),  
    const int sources[](in),  
    const int sourceweights[](in),  
    int outdegree (in),  
    const int destinations[](in),  
    const int destweights[] (in),  
    MPI_Info info(in),  
    int reorder(in),  
    MPI_Comm *comm_dist_graph (out)  
);
```

`reorder` boolean, *true* or *false*, to specify if ranks may be reordered or not

Neighborhood Graph Topology creat.



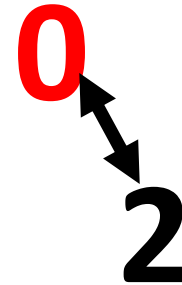
```
int MPI_Dist_graph_create_adjacent (  
    MPI_Comm comm_old(in),  
    int indegree(in),  
    const int sources[](in),  
    const int sourceweights[](in),  
    int outdegree (in),  
    const int destinations[](in),  
    const int destweights[] (in),  
    MPI_Info info(in),  
    int reorder(in),  
    MPI_Comm *comm_dist_graph (out)  
);
```

Comm_dist_graph is the new
communicator embedding the topology

MPI_Dist_graph_create_adjacent ex.

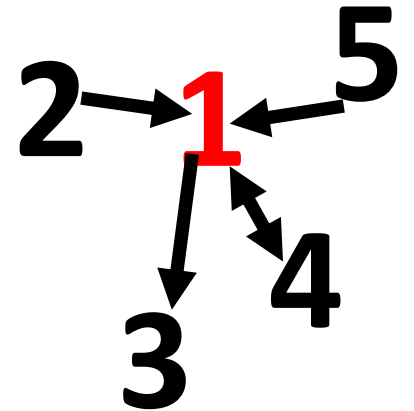
- Rank 0:

- int sources[1] = {2};
- int destinations[1] = {2};



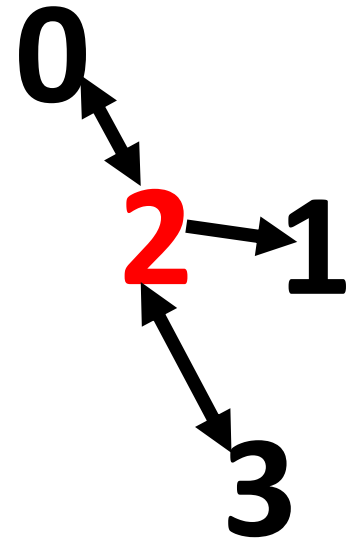
MPI_Dist_graph_create_adjacent ex.

- Rank 0:
 - `int sources[1] = {2};`
 - `int destinations[1] = {2};`
- Rank 1:
 - `int sources[3] = {2,4,5};`
 - `int destinations[2] = {3,4};`



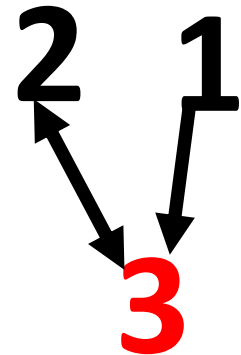
MPI_Dist_graph_create_adjacent ex.

- Rank 0:
 - `int sources[1] = {2};`
 - `int destinations[1] = {2};`
- Rank 1:
 - `int sources[3] = {2,4,5};`
 - `int destinations[2] = {3,4};`
- Rank 2:
 - `int sources[2] = {0,3};`
 - `int destinations[3] = {0,1,3};`



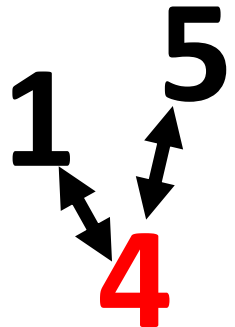
MPI_Dist_graph_create_adjacent ex.

- Rank 0:
 - `int sources[1] = {2};`
 - `int destinations[1] = {2};`
- Rank 1:
 - `int sources[3] = {2,4,5};`
 - `int destinations[2] = {3,4};`
- Rank 2:
 - `int sources[2] = {0,3};`
 - `int destinations[3] = {0,1,3};`
- Rank 3:
 - `int sources[2] = {2,1};`
 - `int destinations[1] = {2};`



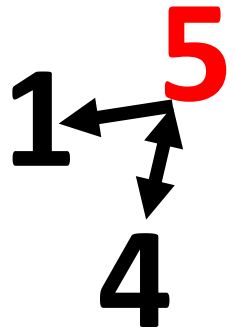
MPI_Dist_graph_create_adjacent ex.

- Rank 0:
 - `int sources[1] = {2};`
 - `int destinations[1] = {2};`
- Rank 1:
 - `int sources[3] = {2,4,5};`
 - `int destinations[2] = {3,4};`
- Rank 2:
 - `int sources[2] = {0,3};`
 - `int destinations[3] = {0,1,3};`
- Rank 3:
 - `int sources[2] = {2,1};`
 - `int destinations[1] = {2};`
- Rank 4:
 - `int sources[2] = {1,5};`
 - `int destinations[2] = {1,5};`



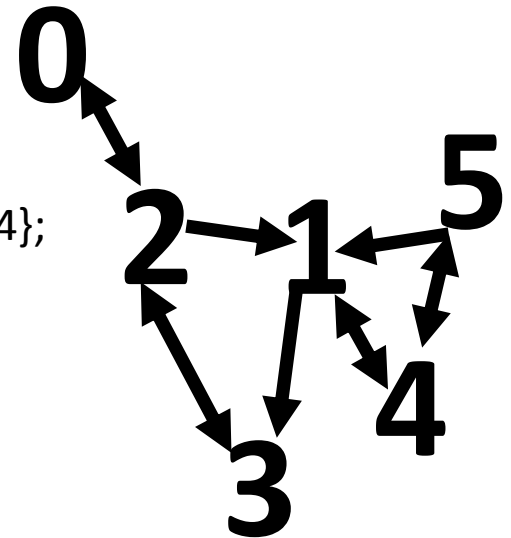
MPI_Dist_graph_create_adjacent ex.

- Rank 0:
 - `int sources[1] = {2};`
 - `int destinations[1] = {2};`
- Rank 1:
 - `int sources[3] = {2,4,5};`
 - `int destinations[2] = {3,4};`
- Rank 2:
 - `int sources[2] = {0,3};`
 - `int destinations[3] = {0,1,3};`
- Rank 3:
 - `int sources[2] = {2,1};`
 - `int destinations[1] = {2};`
- Rank 4:
 - `int sources[2] = {1,5};`
 - `int destinations[2] = {1,5};`
- Rank 5:
 - `int sources[1] = {4};`
 - `int destinations[2] = {1,4};`



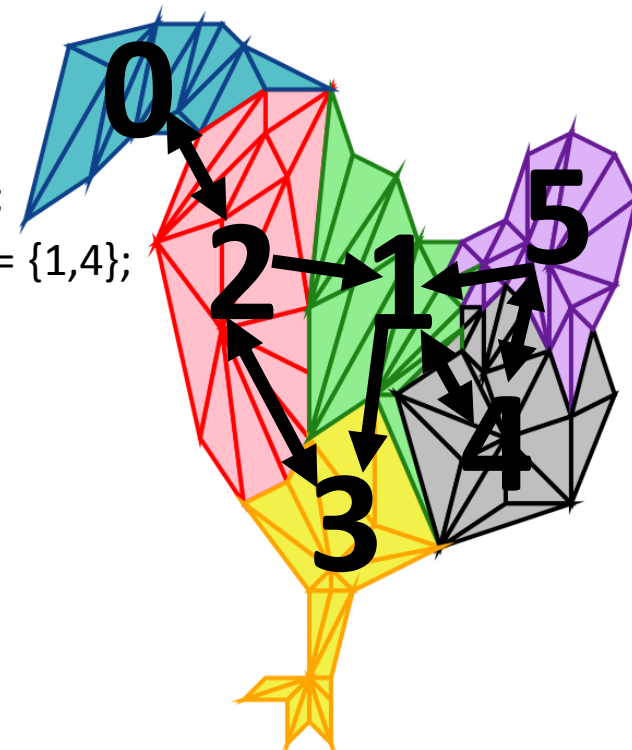
MPI_Dist_graph_create_adjacent ex.

- Rank 0:
 - `int sources[1] = {2};`
 - `int destinations[1] = {2};`
- Rank 1:
 - `int sources[3] = {2,4,5};`
 - `int destinations[2] = {3,4};`
- Rank 2:
 - `int sources[2] = {0,3};`
 - `int destinations[3] = {0,1,3};`
- Rank 3:
 - `int sources[2] = {2,1};`
 - `int destinations[1] = {2};`
- Rank 4:
 - `int sources[2] = {1,5};`
 - `int destinations[2] = {1,5};`
- Rank 5:
 - `int sources[1] = {4};`
 - `int destinations[2] = {1,4};`
- Topology:



MPI_Dist_graph_create_adjacent ex.

- Rank 0:
 - `int sources[1] = {2};`
 - `int destinations[1] = {2};`
- Rank 1:
 - `int sources[3] = {2,4,5};`
 - `int destinations[2] = {3,4};`
- Rank 2:
 - `int sources[2] = {0,3};`
 - `int destinations[3] = {0,1,3};`
- Rank 3:
 - `int sources[2] = {2,1};`
 - `int destinations[1] = {2};`
- Rank 4:
 - `int sources[2] = {1,5};`
 - `int destinations[2] = {1,5};`
- Rank 5:
 - `int sources[1] = {4};`
 - `int destinations[2] = {1,4};`
- Topology:



Dist Graph Topology creation



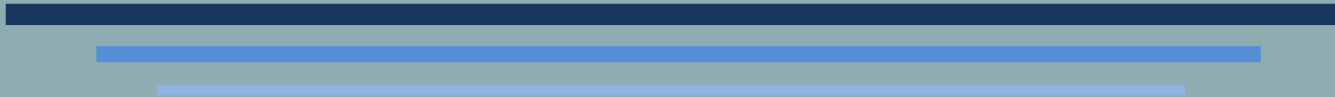
- `MPI_Dist_graph_create(MPI_Comm comm_old, int n, const int sources[], const int degrees[], const int destinations[], const int weights[], MPI_Info info, int reorder, MPI_Comm *comm_dist_graph);`
- Combine both of the two previous graph topology constructor
 - Each MPI rank specify only a subpart of the topology
 - May be any subpart, not only subparts where the current MPI rank is involved
 - Same edges may be specified several times by different MPI ranks

MPI Topology test



- `MPI_Topo_test(MPI_Comm comm, int *status);`
 - IN comm communicator (handle)
 - OUT status topology type of communicator comm (state)
- This function returns in *status* the type of the topology associated with communicator *comm*
- Returned value in status may be :
 - `MPI_GRAPH`
 - `MPI_CART`
 - `MPI_DIST_GRAPH`
 - `MPI_UNDEFINED`

MPI NEIGHBORHOOD COLLECTIVES



MPI Neighborhood collectives



- Definition

- The MPI Neighborhood collectives are collective communications that apply on a communicator with a topology
- Only collectives considered are All-to-All collectives with no operations: Allgather and Alltoall
 - And there derivatives:
 - Allgatherv, Alltoallv, Alltoallw
 - And there nonblocking equivalent
 - lallgather, lallgatherv, lalltoall, lalltoallv, lalltoallw
- Same rules as for other collective communications
- Same function name prefixed with “Neighborhood”

Allgather PP-C2



```
int MPI_Allgather (
```

```
    void *sendbuf(in),
```

```
    int sendcount(in),
```

```
    MPI_Datatype sendtype(in),
```

```
    void *recvbuf(out),
```

```
    int recvcount(in),
```

```
    MPI_Datatype recvtype(in),
```

```
    MPI_Comm comm(in),
```

```
);
```

} Arguments corresponding to
sent data

} Arguments corresponding to
received data



Neighborhood Allgather



```
int MPI_Neighborhood_allgather (
```

```
void *sendbuf(in),
```

```
int sendcount(in),
```

```
MPI_Datatype sendtype(in),
```

```
void *recvbuf(out),
```

```
int recvcount(in),
```

```
MPI_Datatype recvtype(in),
```

```
MPI_Comm comm(in),
```

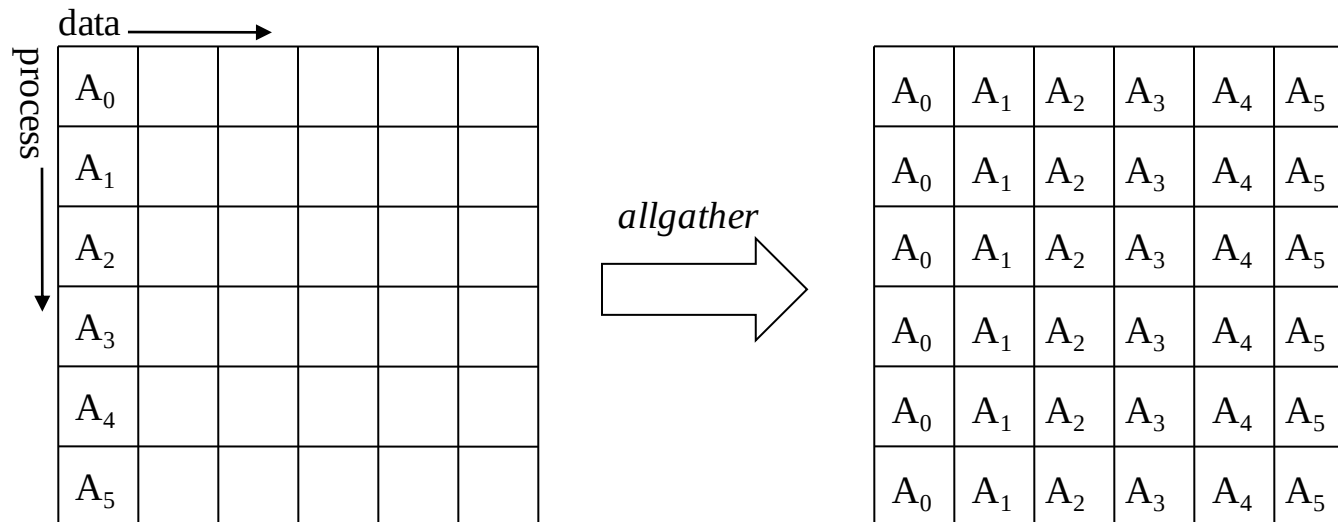
```
) ;
```

} Arguments corresponding to
sent data

} Arguments corresponding to
received data

Allgather PP-C2

- Equivalent to gather collective operation except that every process involved inside the communication receives the final result.
- Allgather $\approx$ Gather + Broadcast





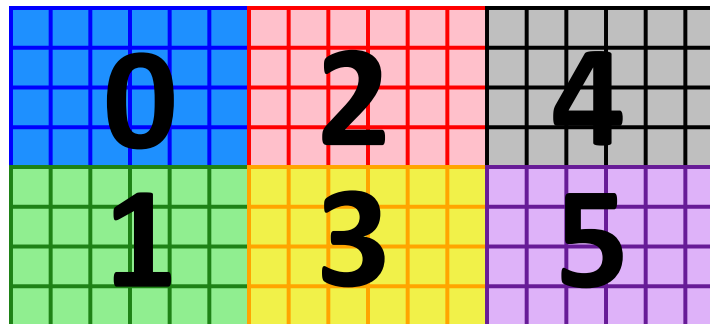
Neighborhood Allgather



- Equivalent to allgather collective operation except that every process involved inside the communication only receives data from incoming edges.

Neighborhood Allgather

- Equivalent to allgather collective operation except that every process involved inside the communication only receives data from incoming edges.
- Consider our cartesian example with bidirectional edges

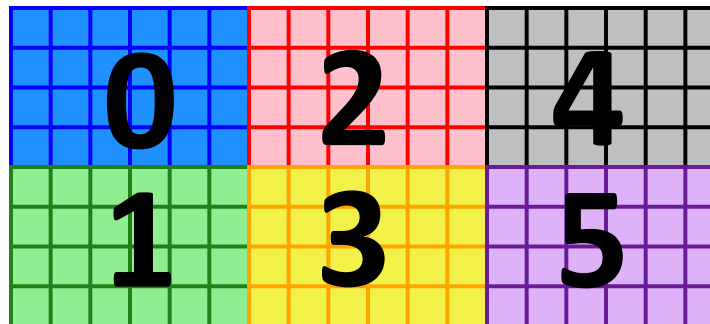




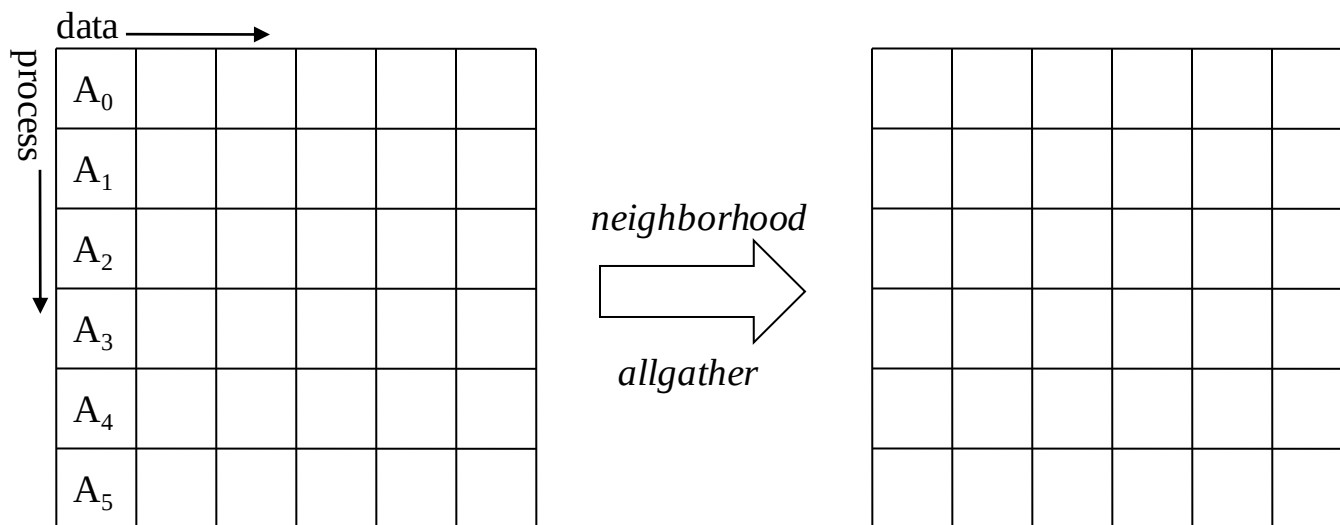
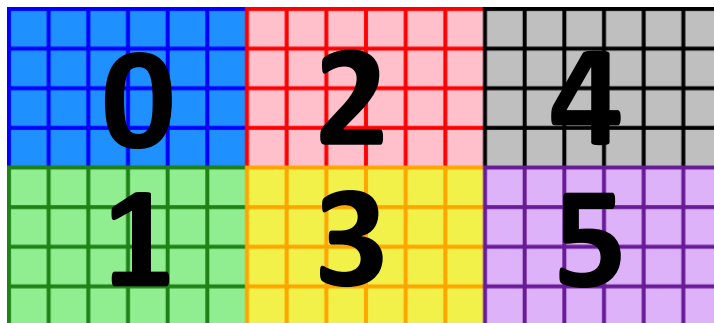
Neighborhood Allgather



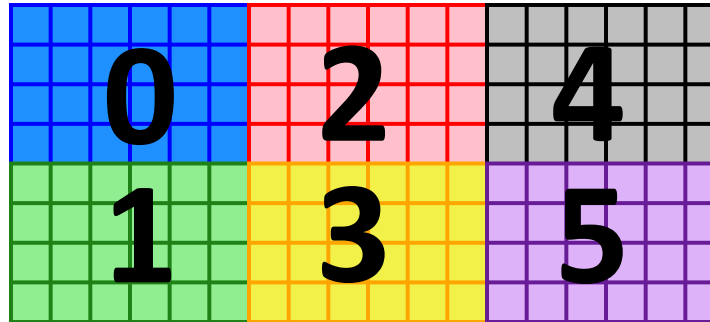
- Equivalent to allgather collective operation except that every process involved inside the communication only receives data from incoming edges.
- Consider our cartesian example with bidirectional edges
- $0 \leftarrow 1,2$ / $1 \leftarrow 0,3$ / $2 \leftarrow 0,3,4$ / $3 \leftarrow 1,2,5$ / $4 \leftarrow 2,5$ / $5 \leftarrow 3,4$



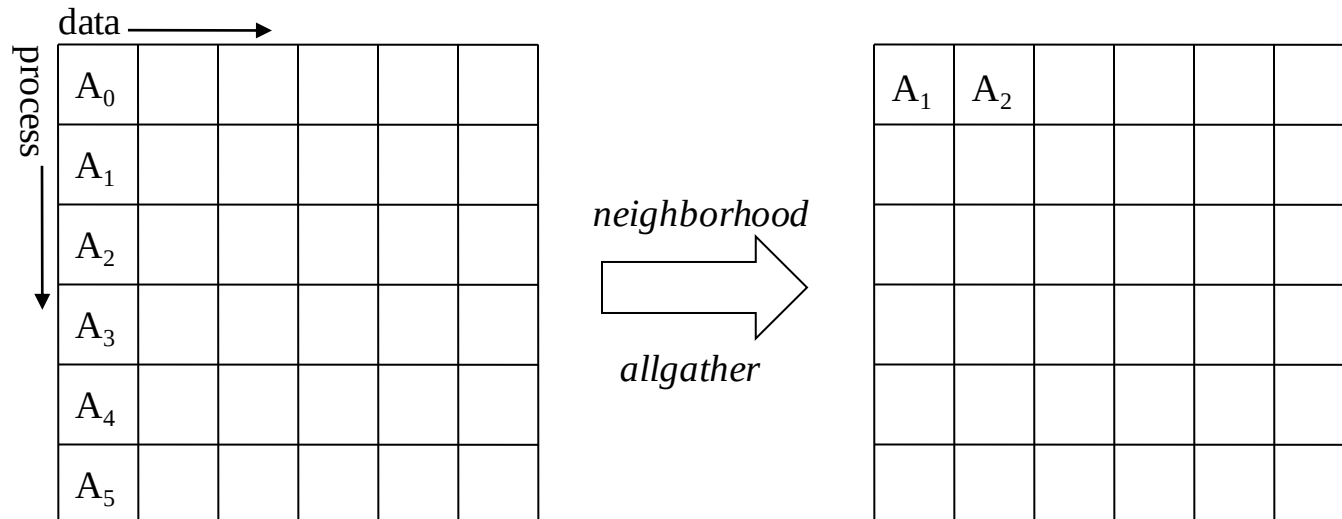
Neighborhood Allgather



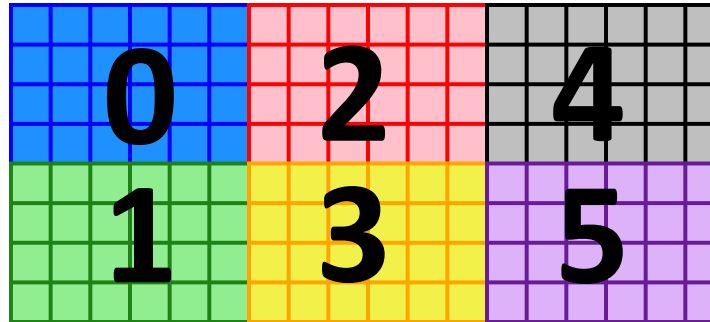
Neighborhood Allgather



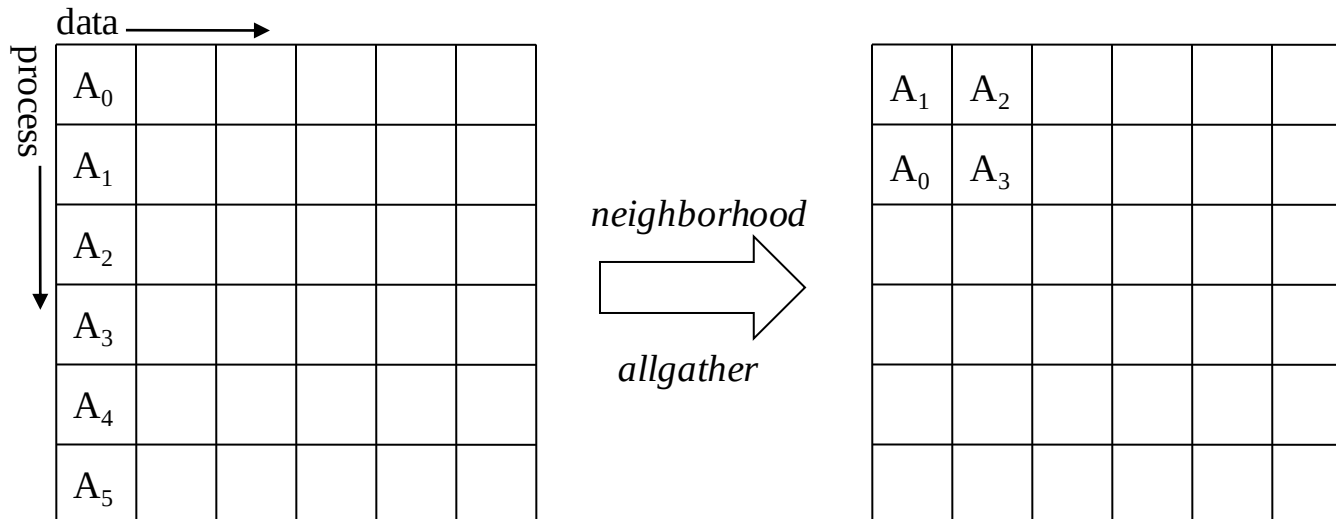
■ 0 < -1,2



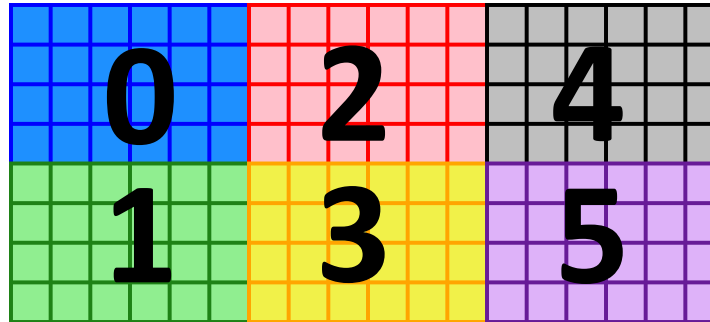
Neighborhood Allgather



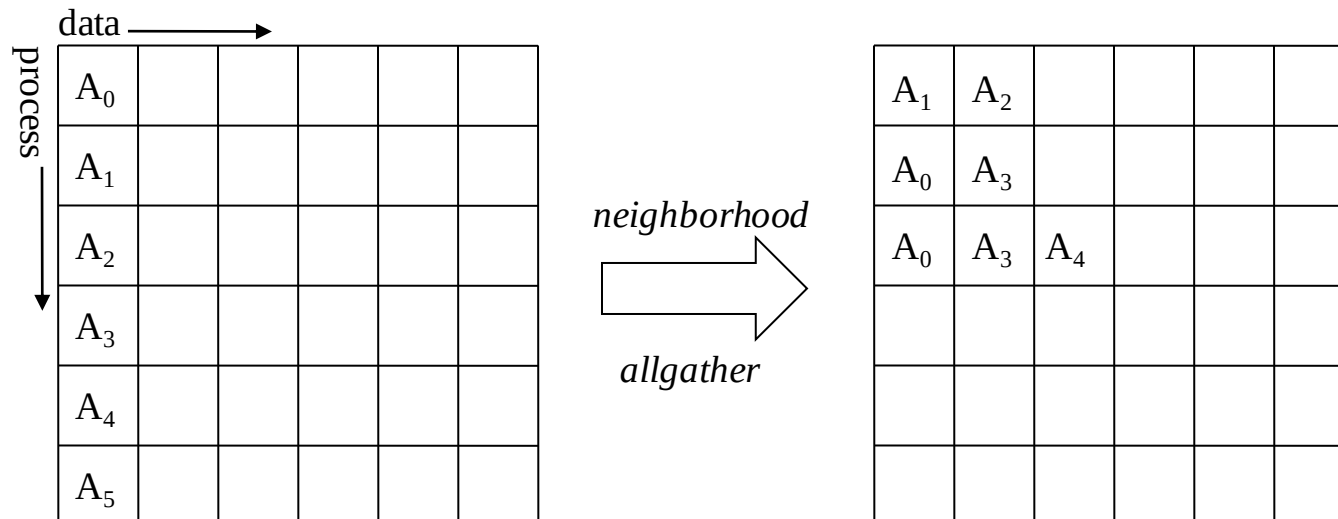
■ $0 < -1, 2 / \underline{1 < -0, 3}$



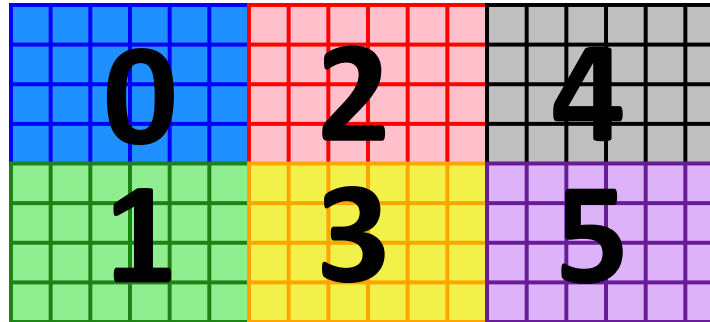
Neighborhood Allgather



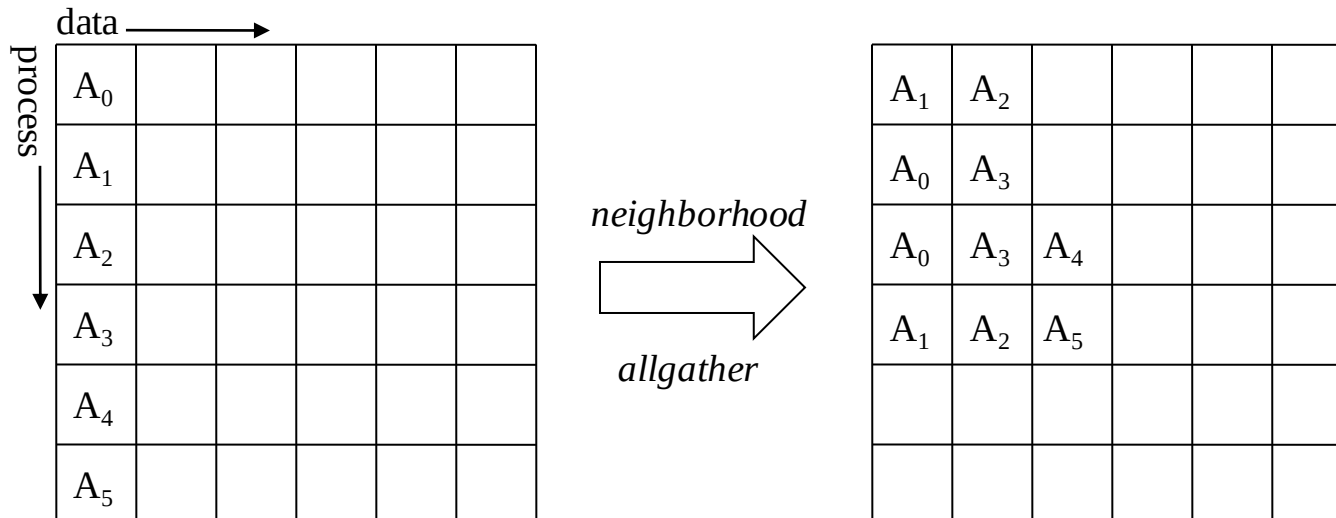
- $0 < -1, 2 / 1 < -0, 3 / \underline{2 < -0, 3, 4}$



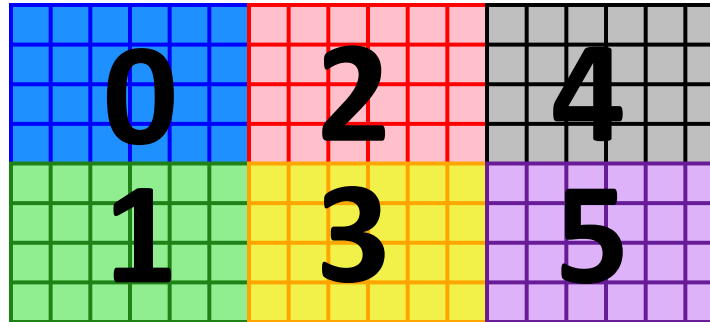
Neighborhood Allgather



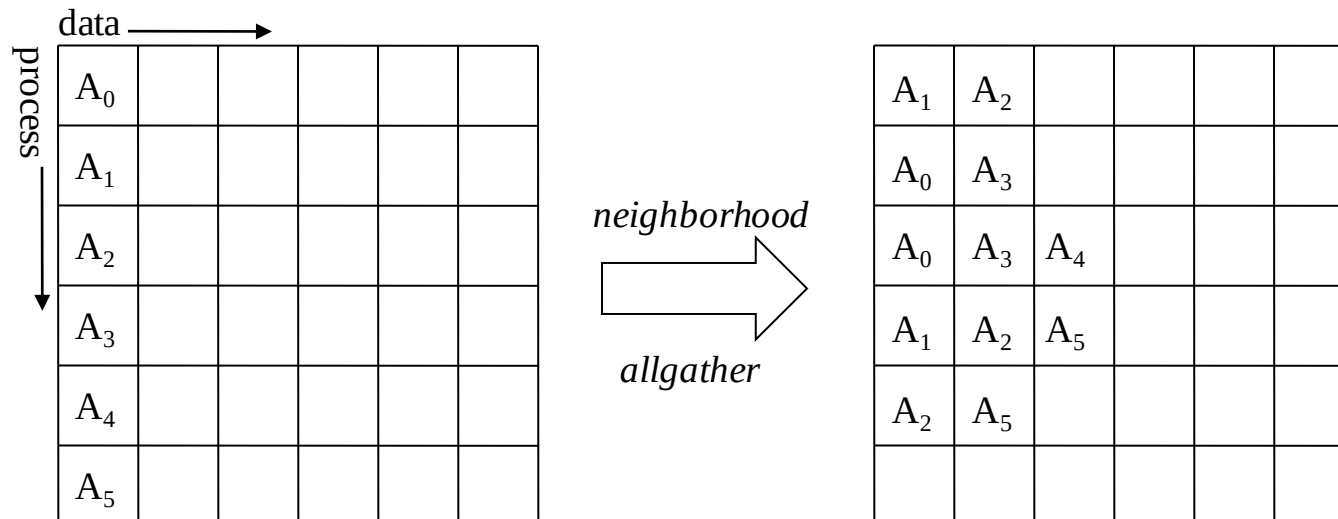
- $0 < -1, 2$ / $1 < -0, 3$ / $2 < -0, 3, 4$ / $3 < -1, 2, 5$



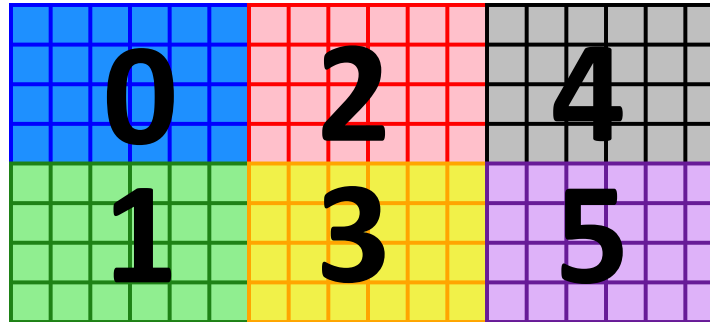
Neighborhood Allgather



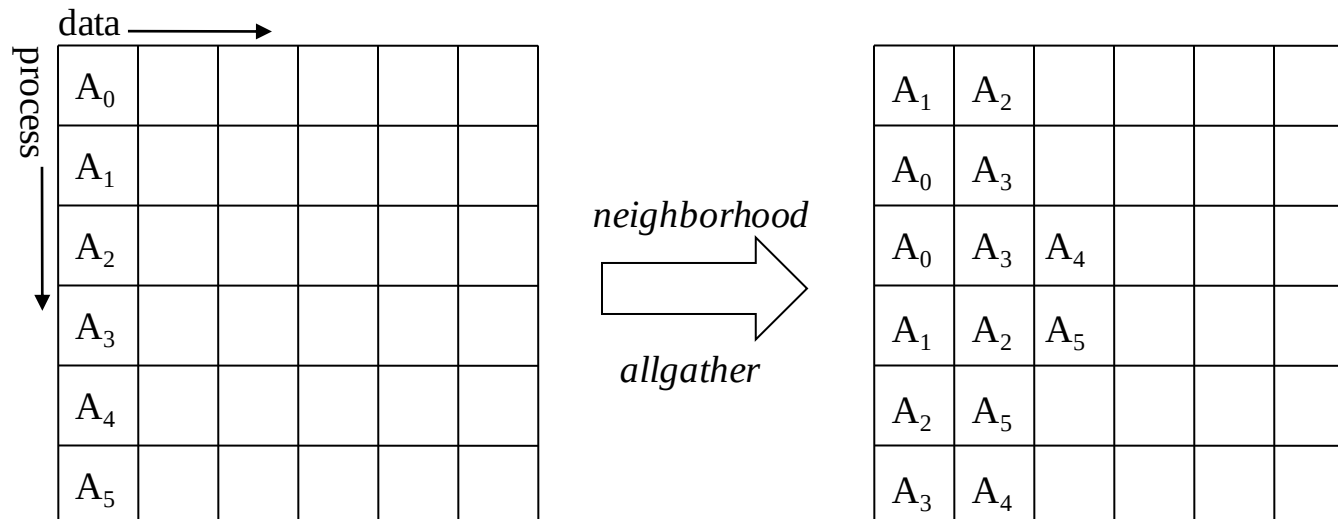
- 0 < -1,2 / 1 < -0,3 / 2 < -0,3,4 / 3 < -1,2,5 / 4 < -2,5



Neighborhood Allgather



- 0 < -1,2 / 1 < -0,3 / 2 < -0,3,4 / 3 < -1,2,5 / 4 < -2,5 / 5 < -3,4



Alltoall PP-C2



```
int MPI_Alltoall (
```

```
    void *sendbuf(in),
```

```
    int sendcount(in),
```

```
    MPI_Datatype sendtype(in),
```

```
    void *recvbuf(out),
```

```
    int recvcount(in),
```

```
    MPI_Datatype recvtype(in),
```

```
    MPI_Comm comm(in),
```

```
);
```

} Arguments corresponding to
sent data

} Arguments corresponding to
received data

Neighborhood Alltoall



```
int MPI_Neighborhood_Alltoall (
```

```
    void *sendbuf(in),
```

```
    int sendcount(in),
```

```
    MPI_Datatype sendtype(in),
```

```
    void *recvbuf(out),
```

```
    int recvcount(in),
```

```
    MPI_Datatype recvtype(in),
```

```
    MPI_Comm comm(in),
```

```
);
```

} Arguments corresponding to
sent data

} Arguments corresponding to
received data

Neighborhood Alltoall

- Equivalent to `MPI_Neighborhood_allgather` collective operation except that every process sends different data to each destination.
- `Neighborhood_allgatherv` and `Neighborhood_alltoallv` are doing the same communication pattern, except that you can specify different displacement in the receive buffer
 - Like usual `allgather` and `alltoall`
- `Neighborhood_alltoallw` is doing the same communication pattern, except that you can specify different displacement and different types for each send/recv
 - Like usual `alltoallw`